Survey of Commodity Price Forecasting

Contents

1 Introduction and Overview

1.1 Introduction and Purpose of the Survey

1.1.1 why commodity price forecasting matters

Price forecasting in agriculture is vital due to its direct impact on livelihoods, food security, and policymaking.

- In agriculture, forecasts go beyond technical accuracy—they guide strategic decisions across multiple stakeholders.
- Farmers use price forecasts to inform crop selection, input allocation, and marketing strategies.
- Forecasts help farmers decide what to grow, when to harvest, and how to sell for optimal returns.
- Governments rely on forecasts for setting minimum support prices (MSPs), managing reserves, and planning trade interventions.
- Predictive insights help stabilize domestic markets through informed policy design.
- Agribusinesses and traders use forecasts for investment planning, logistics, and inventory management.

1.1.2 Purpose of the Survey

The complexity of agricultural markets makes price forecasting especially challenging. Each approach brings strengths and weaknesses depending on data availability and context.

- This survey reviews and compares commodity price forecasting methods in a comprehensive and structured way.
- Both classical econometric models and machine learning techniques are included.
- The review includes the entire forecasting pipeline from feature selection to model training.
- The downstream applications of forecasts are also explored, including risk management and policy evaluation.
- A comparative analysis identifies when each modeling approach is most appropriate.
- This guidance is aimed at helping researchers, policymakers, and practitioners make informed methodological choices.
- The ultimate goal is to enhance decision-making across agriculture sectors, improve food security, and support economic resilience.

1.2 Scope of the report

1.2.1 Market Scope

- Commodity markets fall within the primary sector and include:
  - Agriculture
  - Energy
  - Metals
- This survey concentrates on agricultural commodities due to their:
  - Relevance to food security
  - Importance for farmer livelihoods
  - Sensitivity to public policy
- Agricultural commodities present unique modeling challenges due to:
  - Strong seasonality
  - Vulnerability to supply shocks
  - High policy sensitivity
- Both market types are considered:
  - **Spot markets:** Reflect immediate delivery prices and focus on short-term volatility.
  - **Derivatives markets (futures, forwards, options):** Capture expectations and support hedging, requiring models that incorporate storage costs and contract structures.

1.2.2 Commodity Scope

- Focus is on widely traded, economically significant agricultural commodities, including:
  1. Staple grains: wheat, corn, rice
  2. Oilseeds and derivatives: soybean, groundnut, rapeseed, soya oil
  3. Export/industrial crops: coffee, cotton seed, castor seed, mustard seed, guar seed
- Selection criteria include:
  - Importance in global food systems
  - Trade volumes
  - Policy relevance
  - Data availability
- Key distinction between:

  - **Perishable commodities (e.g., fruits, dairy, meat):** Cannot be stored long-term; exhibit high short-term volatility and strong seasonality.
  - **Storable commodities (e.g., grains, oilseeds):** Can be inventoried; allow intertemporal arbitrage and tighter spot–futures linkages.

- Modeling implications:
  - Perishables require models that capture rapid fluctuations and seasonal patterns.
  - Storables benefit from incorporating inventory dynamics and long-term trends.

1.2.3 Implications for Forecasting Models

- The survey focuses on agricultural commodities across:
  - Spot vs. futures markets
  - Perishable vs. storable goods
- These distinctions affect:
  1. Data availability and frequency
  2. Temporal dependence structures
  3. Role of exogenous variables (e.g., weather, policy)
- Forecasting models must align with:
  - The assumptions used in econometric models
  - Feature selection and training strategies in machine learning models
- The defined scope enables structured comparison of methods across diverse, practically relevant contexts.

2 The Data Landscape

This section catalogs the diverse datasets employed across the surveyed studies, organized by data type and source. The datasets span futures markets, spot markets, fundamental supply–demand indicators, weather and climate variables, remote-sensing products, and macroeconomic covariates.

2.1 Primary commodity and energy datasets

2.1.1 Futures markets

a. (Nishant) Daily agricultural futures price series:
   - Grains: corn, wheat, oats
   - Oilseeds: soybeans, soybean oil, soybean meal

b. (Nishant) Multi-asset daily panel (Jan 2000–Jun 2020; 5,174 observations):

- Agricultural: corn, soybeans, hard wheat, spring wheat
- Livestock: feeder cattle, live cattle, milk
- Energy: WTI crude oil, gasoline
- Financial: gold, U.S. Dollar index, S&P 500, Nasdaq 100

c. (Richard) Futures vs. spot

- Futures markets participants trade standardized contracts (fixed quantity and type of underlying asset), and trading happens on a central exchange (e.g. CME)
- Spot transactions happen directly between two counterparties, and they are usually customized and private (OTC, over-the-counter)
- Futures contracts obligate parties to buy or sell an underlying asset at a pre-determined price and expiration date (maturity) in the future, while spot price is the price for buying the underlying assets right now
- F = S * exp((r+c-i)*t)
  - F is futures price
  - S is spot price
  - r is risk-free rate
  - c is cost of storage of asset
  - i is convenience yield (the benefit of holding the asset)
  - t is time to maturity
  - c and i need to be estimated, reflecting people's expectation for the future
- Futures price can be higher than spot price (contango) or lower than spot price (backwardation)
- Futures price doesn't always move in the same direction or with the same magnitude as spot price
- To enter a position in the futures market, the trader needs to put down a deposit/margin equal to a few percentages of the current futures price (leveraged)
- Futures positions are revalued daily; gains/losses are settled each day through margin accounts
- Futures contracts on the same commodity can have different expiration dates, and the contract with the closest expiration date is called the front-month contract
- Front-month contract usually has the highest liquidity
- Each time a front-month contract expires, a new front-month contract replaces it (roll-over)
- Roll-over can cause jumps/gaps in futures prices due to the change in time-to-maturity t (also called roll-over PnL)
- When people mention futures historical price data , they usually refer to the historical prices of the front-month contracts (continuous front-month contract)

2.1.2 Spot and wholesale markets

a. (Peter) Indian vegetable prices:

- Onion (daily; Kurnool market)
- Multi-vegetable panel across Indian markets

b. (Nived) Indian oilseed and fiber crop prices:

- Oilseeds: groundnut, rapeseed, soybean, castor, mustard, guar
- Oil products: soya oil
- Fiber: cotton seed

c. (Nived) Vietnam commodity prices:

- Coffee spot price (daily)
- Fuel prices (diesel, gasoline)

d. (Nishant) Horticulture prices:

- Brinjal/eggplant (daily; 17 markets in Odisha, India; 1 Jan 2015–31 May 2021)
- Asparagus (monthly aggregation)

2.1.3 Fundamental supply–demand data

a. (Nishant) USDA production and balance-sheet data:

- WASDE reports (monthly; production, stocks, consumption)
- ERS Feed Grains Database (annual corn supply–demand; 47-year panel)
- D2-corn series: year, yield, residue, imports
- D3-corn series: year, demand, price

2.1.4 Perishable vs storable commodities (Richard)

a. Soft commodities are grown or harvested and need to be stored in controlled environments, such as coffee, cocoa, sugar, corn, wheat, soybean, fruit and livestock

b. Hard commodities such as crude oil, natural gas, gold, copper, and other metals are extracted/mined, are non-perishable and generally easier to store and transport

c. Soft commodities prices are highly influenced by weather, harvest surprises, climate change, storage cost/capacity, pests, diseases, and supply chain pressures

d. Hard commodities prices tend to reflect industrial demand and macroeconomic conditions

e. Both soft and hard commodities prices can be highly volatile

2.2 Exogenous data sources

2.2.1 Weather and climate

a. (Peter) Local weather for Indian markets (daily):

- Temperature (max/min), humidity, precipitation
- Wind speed, sea level pressure
- Solar radiation, UV index

b. (Nived) Weather for Vietnam coffee regions:

- Temperature, humidity
- Rainfall, seasonal indicators, extreme-weather flags (optional)

2.2.2 Remote sensing (Richard)

a. Landsat satellite imagery (via Google Earth Engine):

- Sensors: Landsat 1–9 (MSS, TM, ETM+, OLI/TIRS)
- Crop-area targeting with temporal alignment (e.g., 20 days pre-prediction)

b. Derived environmental variables:

- Cloud cover, sun elevation/azimuth
- Vegetation indices (optional)

2.2.3 Supply–demand fundamentals

a. (Nishant) Production and stock metrics (crop-specific):

- Production, yield, harvested area
- Beginning/ending stocks, total supply
- Domestic use, feed/crushing demand, exports/imports
- Stock-to-use ratios

2.2.4 Cross-commodity and price spillovers

a. (Nishant) Inter-commodity price linkages:

- Wheat price $\rightarrow$ corn models
- Soybean oil/meal $\rightarrow$ soybean models
- Multi-variate grain/livestock/energy setups

2.2.5 Macroeconomic and financial indicators

a. (Nishant) Macro-financial covariates:

- Equity: S&P 500, Nasdaq 100
- Currency: U.S. Dollar index
- Commodities: gold, WTI crude, gasoline
- Livestock/dairy: feeder cattle, live cattle, milk

b. (Nishant) Macroeconomic determinants:

- Exchange rates
- GDP, macro activity
- Energy prices (cost linkages)

2.2.6 Time-series transformations

a. (Nishant) Engineered temporal features:

- Lagged prices (multiple lag structures)
- Returns, log-returns, price differences
- De-trending, de-seasonalization, differencing

2.3 Data provenance and platform

2.3.1 Academic literature sources

These sources provide access to previous work, benchmark datasets, and methodological comparisons in forecasting.

a. (Peter) Systematic review databases:

- Scopus, Web of Science, Google Scholar

b. (Nishant) Publisher platforms:

- IEEE Xplore, Wiley Online Library, PLOS ONE, ACM Digital Library

2.3.2 Commodity price providers

These platforms provide the core time series of spot and futures prices for agricultural commodities—the primary target variable in forecasting models. Typical frequency is daily or monthly, depending on the provider and product. Data spans range from 5 to 30 years.

a. (Nishant,Peilin) Futures and derivatives:

- Investing.com (daily agricultural futures)
- Barchart (continuous contracts, historical series)
- The Chicago Board of Trade (CBOT)

b. (Nived, Nishant, Peilin) Government portals:

- AGMARKNET (India): wholesale prices for vegetables, oilseeds
- CEPEA/Esalq website(Brazil)
- Eurostat(Euro)
- FAMA official government(Malaysia)

c. (Peter) Public datasets:

- Kaggle (onion prices)

d. (Richard + Nived + Peilin) Other sources:

- Yahoo Finance (unofficial API access)
- TradingViews (high-frequency historical data at a reasonable price)
- Bloomberg Terminal (FRE department free access)
- CME exchange data
- Coffee and Agricultural data from Vietnam Govt website
- Our World in Data

2.3.3 Fundamental and weather data

These datasets provide variables that influence commodity prices but are not themselves forecast targets. They are typically used as features in predictive models. **Supply and demand** reports help anticipate medium-term price movements by identifying structural imbalances. **Weather and environmental data** are essential for modeling short-term yield shocks and seasonality patterns. **Labor utilization data** reflects underlying production capacity and regional shocks, especially in labor-intensive crops.

a. (Nishant) USDA portals:

- WASDE (monthly production reports)
- ERS Feed Grains Database (annual supply/demand)

b. (Peter, Richard, Nived) Environmental data:

- Visual Crossing (daily weather)
- Google Earth Engine (Landsat imagery)
- Open Meteo (weather data)

c. (Peilin) Labor Utilization data:

- the Immigration and Customs Enforcement (ICE-US)

2.3.4 Temporal coverage summary

- Multi-asset daily: Jan 2000–Jun 2020 (5,174 obs.)
- Brinjal (Odisha): 1 Jan 2015–31 May 2021
- WASDE: monthly, aligned to study windows
- ERS corn: 47-year annual panel
- Typical ranges (Peter, Nived): 2009–2022
- Replication availability: ∼2020 onward

3 Feature Engineering and Input

This section organizes the feature types employed across the surveyed literature, ranging from raw price series and fundamental supply–demand indicators to weather variables, technical transformations, and emerging data sources such as satellite imagery and sentiment analysis.

3.1 Market and price features

3.1.1 Core price series

a. Spot and wholesale prices:

- (Peter) Daily onion prices (Kurnool market)
- (Peter) Minimum, maximum, and modal prices for multiple vegetables across Indian markets

b. (Nishant, Nived) Futures price series (daily; continuous/rolled contracts):

- Agricultural: corn, wheat, soybeans, oats, soybean oil, soybean meal
- OHLCV structure: open, high, low, close, volume, percentage change
- Contract metadata: expiration month, roll timing, liquidity proxies (volume, open interest)

c. (Nishant) Multi-asset futures panel (cross-market predictive covariates):

- Grains: corn, soybeans, wheat, oats
- Livestock: feeder cattle, live cattle
- Energy: WTI crude oil, gasoline
- Financial: gold, S&P 500, Nasdaq 100, U.S. Dollar index

d. (Nishant) Annual fundamental-price series (47-year corn panel):

- Yield, end-of-year residue, imports, demand, price

e. (Nishant) Specialty crops:

- Asparagus (monthly; Malaysia)

3.1.2 Market identifiers and cross-sectional structure

- Geographic: state, district, market
- Product: commodity, variety
- (Nishant) Futures contract metadata: contract month, expiration date, roll rules, liquidity metrics

3.1.3 Temporal structure and transformations

a. Basic temporal features:

- Calendar date, timestamp
- Lagged prices (multiple lag structures)
- Returns and differences (simple or logarithmic)
- Rolling statistics: moving averages, volatility, momentum

b. (Nishant) Continuous futures construction:

- Roll rules: fixed days before expiration, based on volume/open interest
- Active contract selection via liquidity proxies

c. (Nishant) Stationarity transformations:

- Full adjustment (FA): log $\rightarrow$ deseason $\rightarrow$ detrend $\rightarrow$ standardize
- First differencing (FD): remove unit roots
- Partial adjustments: deseason-only (SA) or detrend-only (TA)
- Regime-specific modeling: post-structural-break samples

3.1.4 Volume and market activity

- Arrival quantities (spot markets): proxy for short-run supply pressure
- Futures volume and open interest: liquidity indicators, speculative vs. hedging positions

3.2 Weather and environmental features

3.2.1 Local weather variables

a. (Peter) Daily weather for Indian markets:

- Temperature: maximum, minimum
- Precipitation: rainfall totals, probability
- Humidity
- Wind speed, sea level pressure
- Solar radiation, UV index

b. (Nived) Weather for Vietnam coffee regions:

- Temperature, humidity
- Rainfall, seasonal dummies, extreme-weather flags (optional)

3.2.2 Remote-sensing and satellite-derived features

- (Richard) Landsat-derived variables:
  - Cloud cover over cropped areas
  - Sun elevation and azimuth
  - Vegetation/greenness indices (optional)

3.3 Feature families from systematic reviews

The three systematic literature reviews identify the following feature categories as commonly employed in commodity price forecasting:

3.3.1 Price-based (technical) features

- Raw and adjusted prices (real prices, inflation-adjusted)
- Lagged prices and returns (simple or logarithmic) over multiple horizons
- Rolling indicators: moving averages, volatility, high–low spreads, momentum measures

3.3.2 Fundamental supply–demand features

- Production metrics: output, harvested area, yield (national/regional scales)
- Consumption: domestic use, export/import quantities
- Stocks: beginning/ending stocks, stock-to-use ratios (by region or marketing year)

3.3.3 Weather and climate features

- Temperature: averages, anomalies, extreme heat/cold indicators
- Precipitation: totals, deviations from normal, drought/flood indicators
- Soil moisture, drought indices, vegetation/crop condition measures

3.3.4 Cost, macroeconomic, and trade environment

- Production costs: fertilizer, fuel/energy, seed, labor indices
- Macro variables: exchange rates (vs. USD), inflation, interest rates, crude oil, related commodity prices
- Trade flows: export/import volumes, trade balances, freight/shipping costs
- Policy indicators: export bans, support prices, public stock releases, subsidies

3.3.5 Market and financial structure features

- Futures term structure: price curves, basis (futures–spot spread), delivery month spreads
- Trading activity: volume, open interest, speculative vs. hedging positions
- Volatility indicators: derived from futures or options prices

3.3.6 Emerging feature types

- Satellite-based biophysical indices: vegetation indices, greenness, crop condition
- Textual and sentiment data: news articles, official reports, web search, social media (summarized as event/sentiment scores)
- Calendar and institutional timing: planting/harvest windows, seasonal dummies, policy announcement dates

3.4 Feature selection and lag determination

3.4.1 Selection methodologies employed in surveyed studies

a. (Nishant) Correlation-driven selection:
   - Heatmap analysis to identify top predictors (e.g., wheat price, consumption, stock ratios)
   - Retention of "top-k" features based on correlation with target

b. (Nishant) Cointegration-based reduction:
   - Start with large multivariate sets (e.g., 13 series)
   - Retain smaller subsets preserving long-run equilibrium relationships

c. (Nishant) Automated nonlinear lag selection:
   - Differential evolution for lag-time optimization
   - Engineered inputs: squared/scaled lag terms, time-difference terms

d. (Nishant) ACF/PACF-guided lag selection:
   - Use autocorrelation spikes as initial lag range candidates
   - Applied to both classical baselines (ARIMA) and neural network input design

e. (Nishant) Derived feature construction:
   - Supply = predicted yield + residue + imports
   - Demand predicted from strongest time-correlation patterns

3.5 Summary

The systematic reviews reveal that most empirical work centers on price data combined with a limited set of supply–demand and macroeconomic variables. Remote-sensing products and textual/sentiment features are emerging but remain relatively uncommon. The surveyed studies demonstrate diverse approaches to feature engineering, ranging from simple lagged prices to sophisticated transformations, cross-asset spillovers, and automated selection procedures.

4 Downstream Applications and Objectives

This section catalogs the forecasting tasks and decision-support applications addressed across the surveyed literature. While point price prediction dominates, studies increasingly explore multi-horizon forecasting, volatility modeling, and operational decision support.

4.1 Core forecasting tasks

4.1.1 Price prediction horizons

a. One-step-ahead prediction:

- Forecast next-period price of agricultural commodities (onion, vegetables, grains, oilseeds)
- Framed as supervised regression using historical prices and covariates
- Dominant objective across both focal studies and systematic reviews

b. (Nishant) Multi-horizon forecasting:

- Short multi-step horizons (days to weeks ahead)
- Aggregated forecasts over operational windows (weekly, monthly summaries)
- Aligned with reporting cycles and decision timelines

4.1.2 Market types and forecast targets

a. Spot and wholesale market prices:

- (Peter, Nived) Daily prices for vegetables, oilseeds, coffee
- (Nishant) Horticulture products (brinjal, asparagus)

b. (Nishant) Futures contract prices:

- Agricultural futures: corn, soybeans, wheat, oilseed products
- Market direction/trend classification alongside point forecasts

c. (Nishant) Ensemble and composite forecasts:

- Combine econometric and machine learning models
- Improve robustness across regimes and market conditions

4.2 Secondary forecasting tasks

Beyond price prediction, the systematic reviews identify several related modeling objectives:

a. Yield and production forecasting:
   - Crop yields and production volumes as inputs to price models
   - (Nishant) Integrated with supply construction pipelines

b. Volatility and risk modeling:
   - Price volatility dynamics
   - Distributional properties and risk measures (e.g., VaR)
   - Used for risk management and hedging strategy design

c. Spillover and connectedness analysis:
   - Cross-commodity linkages
   - Spot-futures relationships
   - Spatial spillovers and lead-lag effects across regions

d. (Nishant) Structural supply-demand-price pipelines:
   - End-to-end modeling: yield $\rightarrow$ supply $\rightarrow$ demand $\rightarrow$ price
   - Supply construction: predicted yield + residue + imports
   - Support profit/loss estimation beyond pure forecast accuracy

4.3 Decision support applications

4.3.1 Operational and trading decisions

a. (Peter) Market-crop allocation support:
   - Identify which markets demand which crops
   - Guide supply shipment and prioritization decisions

b. (Nishant) Market selection and switching:
   - Choose optimal selling location among nearby markets
   - Particularly relevant for perishable vegetables

c. Agricultural timing decisions:
   - Planting, storage, and sales timing under price uncertainty
   - (Nishant) Harvest timing and sell decisions based on price trajectories

d. (Nishant) Commodity trading and risk management:
   - Real-time market analysis

- Trading strategy support for suppliers and intermediaries
- Portfolio hedging applications

e. (Nishant) Farmer profit/loss estimation:

- Translate supply-demand-price forecasts into profit expectations
- Inform production and marketing decisions

4.3.2 Policy and planning applications

a. (Nishant) Policy intervention and planning:

- Early preparation under climate, exchange rate, or energy price shocks
- Control policy design and plantation planning adjustments

b. Food security and price stabilization:

- Support public stock management decisions
- Guide subsidy and support price policies

4.3.3 Anomaly detection and quality control

a. (Richard) VAE-based anomaly detection:

- Reconstruct price and volume time series
- Flag high reconstruction error as potential anomalies
- Identify data errors, extreme shocks, structural breaks

b. Data quality monitoring:

- Detect reporting errors in wholesale market data
- Identify outliers and unusual price movements

4.4 Summary

The surveyed literature demonstrates diverse applications of commodity price forecasting. While one-step-ahead price prediction remains the dominant task, studies increasingly address multi-horizon forecasting, volatility modeling, and operational decision support. Applications span farmer-level decisions (market selection, harvest timing), trader and intermediary functions (trading strategies, risk management), and policy planning (intervention design, food security). Emerging work explores anomaly detection and integrated supply-demand-price pipelines that support profit/loss estimation beyond pure forecast accuracy.

5 Model Taxonomy and Comparison

5.1 Deep learning and neural-network models

5.1.1 Overall role

- Most frequently used modern models in the 3 review papers.
- Often applied after a decomposition step (e.g., EMD/CEEMDAN + DL).

5.1.2 Core architectures

ANN (Artificial Neural Network) → Peilin Yao

- **Conceptual Overview**
  - Feedforward neural network that learns a nonlinear mapping from inputs to outputs.
  - Assumes a stable relationship between features and target within the training distribution.
  - Does not explicitly model time or state unless lagged inputs are manually provided.
- **Implications for Time Series and Non-Stationarity**
  - Performs well when price dynamics are relatively stable over time.
  - Struggles under regime shifts because learned weights encode past regimes.
  - Requires retraining or rolling windows to adapt to structural changes.
- **Model Structure**
  - Input layer receives feature vector (e.g., lagged prices, exogenous variables).
  - Hidden layers capture nonlinear interactions between features.
  - Output layer produces point forecasts for regression tasks.
- **Forward Propagation**
  - Pre-activation: $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$
  - Activation: $a^{(l)} = \sigma(z^{(l)})$
  - Output: $\hat{y} = a^{(L)}$
- **Training Objective**
  - Loss (regression): $\mathcal{L} = \frac{1}{N}\sum(y_i - \hat{y}_i)^2$
  - Parameters updated via backpropagation and gradient descent.
- **Strengths**
  - Flexible nonlinear function approximation.
  - Handles high-dimensional and multivariate inputs.

- **Limitations**
  - No internal memory or temporal state.
  - Sensitive to non-stationarity and regime changes.
  - Requires large datasets and careful regularization.

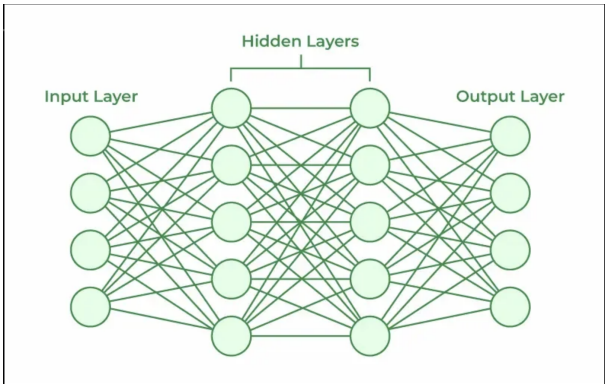

Figure 1: Artificial Neural Network Architecture

ELM (Extreme Learning Machine) → Peilin Yao

- **Conceptual Overview**
  - Single-hidden-layer neural network with randomly fixed hidden parameters.
  - Only output weights are learned, using a closed-form solution.
  - Designed for fast training rather than sequential modeling.
- **Implications for Time Series and Non-Stationarity**
  - Adapts quickly to new regimes due to extremely fast retraining.
  - Does not internally model temporal dependence or state.
  - Performance depends on feature engineering to encode time structure.
- **Model Structure**
  - Input layer accepts feature vector $x \in \mathbb{R}^n$.
  - Hidden layer uses random weights W and biases b with activation $g(\cdot)$.
  - Output layer computes linear combination of hidden activations.
- **Hidden Layer Transformation**
  - Neuron output: $h_j(x) = g(w_j \cdot x + b_j)$
  - Hidden matrix: $H \in \mathbb{R}^{N \times L}$
- **Output Weight Estimation**
  - Closed-form solution: $\beta = H^{\dagger}T$
  - Regularized form: $\beta = (H^{\top}H + \lambda I)^{-1}H^{\top}T$

- **Prediction**
  - $f(x) = H\beta$
- **Strengths**
  - Extremely fast training.
  - Avoids local minima.
  - Suitable for frequent retraining under regime shifts.
- **Limitations**
  - No memory or sequential structure.
  - Sensitive to number of hidden neurons.
  - Limited interpretability.

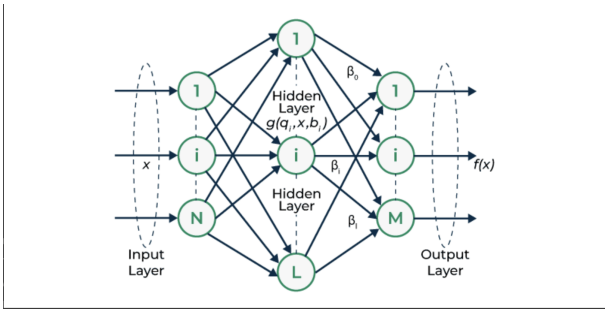

Figure 2: Extreme Learning Machine Architecture

RNN (Recurrent Neural Network) → Peilin Yao

- **Conceptual Overview**
  - Designed to model sequential data by maintaining an internal hidden state.
  - Captures temporal dependence by feeding past information forward in time.
  - Suitable when current prices depend on recent historical behavior.
- **Information Flow and Memory**
  - Uses shared weights across time steps to process sequences.
  - Maintains a hidden state h_t that summarizes past inputs.
  - Memory is implicit and compressed into a single state vector.
- **Implications for Non-Stationarity and Regime Shifts**
  - Performs well when temporal dependencies are stable over time.
  - Struggles under regime shifts because past states dominate current predictions.
  - Long-term dependencies may be distorted due to vanishing or exploding gradients.

  - Requires frequent retraining or truncated memory for evolving markets.

- **State Update Mechanism**
  - Hidden state combines current input and previous state:
$$h_t = \sigma(Ux_t + Wh_{t-1} + B)$$
  - U controls input influence; W controls temporal persistence.

- **Output Generation**
  - Output derived directly from hidden state:
$$y_t = O(Vh_t + C)$$
  - Output reflects both current input and historical context.

- **Recursive Representation**
  - General formulation:
$$h_t = f(h_{t-1}, x_t)$$
  - Example using tanh activation:
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$y_t = W_{hy}h_t$$

- **Training via Backpropagation Through Time (BPTT)**
  - Gradients propagate backward across time steps.
  - Weight updates accumulate effects from all previous states.
  - Simplified gradient form:
$$\frac{\partial L}{\partial W} = \sum_k \frac{\partial L}{\partial h_t}\frac{\partial h_t}{\partial h_k}\frac{\partial h_k}{\partial W}$$

- **Strengths**
  - Explicit temporal modeling.
  - Effective for short- to medium-term dependencies.

- **Limitations**
  - Vulnerable to vanishing and exploding gradients.
  - Poor long-term memory compared to gated architectures.
  - Limited robustness to abrupt regime changes.

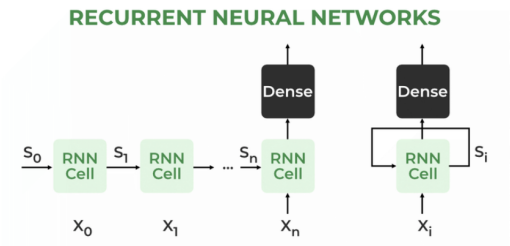

Figure 3: Recurrent Neural Network Architecture

ESN (Echo State Network) → Peilin Yao

- **Conceptual Overview**
  - ESNs are a class of recurrent networks with fixed internal dynamics.
  - Only the output weights are trained, enabling fast learning on sequential tasks.
  - The "echo" refers to the fading memory of past inputs in the reservoir state.
- **Memory and Dynamics**
  - A large, randomly connected reservoir stores a nonlinear transformation of input history.
  - Dynamics are driven by the input and previous state; no gradients are propagated through time.
  - Stability is controlled via the **Echo State Property**.
- **Implications for Non-Stationarity and Regime Shifts**
  - ESNs adapt well to dynamic patterns with short-to-medium temporal dependencies.
  - Fast retraining makes them suitable for shifting regimes.
  - However, random reservoir initialization can make performance unstable across domains.
  - Memory capacity is fixed and bounded by reservoir size and spectral radius.
- **Reservoir State Update**
  - Current state depends on previous state and current input:

$$x(t) = (1-\alpha)x(t-1) + \alpha \cdot f\left(W^{\text{in}}u(t) + Wx(t-1) + W^{\text{fb}}y(t-1)\right)$$

  - $W^{\text{in}}, W, W^{\text{fb}}$ are fixed random weights; α is the leaking rate.
  - f is a nonlinear activation (e.g., tanh).
- **Echo State Property (ESP)**
  - Ensures that the effect of past inputs decays over time:

$$\rho(W) < 1$$

  - $\rho(W)$ is the spectral radius of the reservoir weight matrix.
- **Output Computation**
  - Output is a linear readout from the concatenated input and reservoir state:

$$y(t) = W^{\text{out}}[u(t); x(t)]$$

  - Only W^{out} is learned.
- **Training via Ridge Regression**

- Collect hidden states into matrix X, targets into Y.
- Output weights are solved as:

$$W^{\text{out}} = YX^{\top}(XX^{\top} + \lambda I)^{-1}$$

- λ is a regularization term.

- **Strengths**
  - Fast training — only output weights are updated.
  - No vanishing gradient problem.
  - Captures nonlinear dynamics with low computational cost.

- **Limitations**
  - Performance sensitive to hyperparameters (reservoir size, spectral radius).
  - Requires multiple trials due to randomness.
  - Theoretical understanding of optimal reservoir design is limited.
  - Memory limited by reservoir expressiveness.

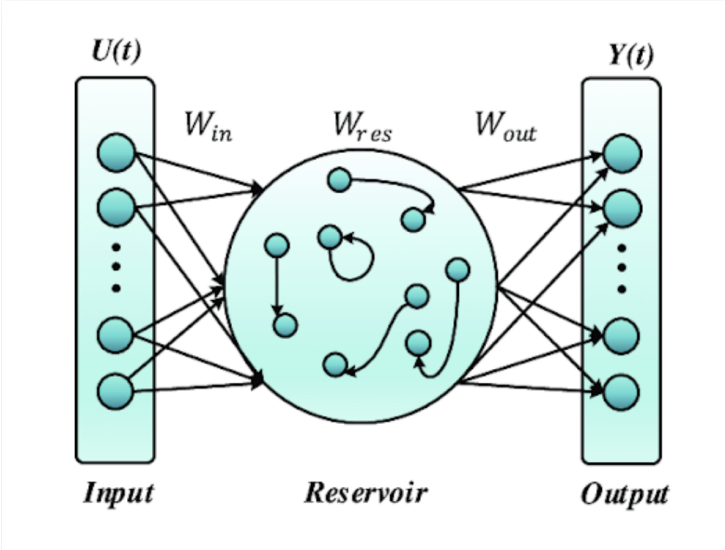

Figure 4: Echo State Network Architecture showing the input layer, randomly connected reservoir, and trained output layer.

LSTM (Long Short-Term Memory networks) → Nived

i. The Long Short-Term Memory (LSTM) network is a special type of RNN that is designed to handle long-term dependencies in sequential data like time-series of stock or agricultural prices.

ii. It overcomes the vanishing gradient issue seen in standard RNNs.

iii. Each LSTM cell has a memory cell that can remember information for long periods

iv. Pros - captures dependencies, no vanishing gradient problem

v. Cons - more computationally expensive

The LSTM equations are:

$$h_t = \sigma(x_t U^i + h_{t-1} W^i) \tag{1}$$
$$f_t = \sigma(x_t U^f + h_{t-1} W^f) \tag{2}$$
$$o_t = \sigma(x_t U^o + h_{t-1} W^o) \tag{3}$$
$$\tilde{C}_t = \tanh(x_t U^g + h_{t-1} W^g) \tag{4}$$
$$C_t = \sigma(f_t * C_{t-1} + i_t * \tilde{C}_t) \tag{5}$$
$$h_t = \tanh(C_t) * o_t \tag{6}$$

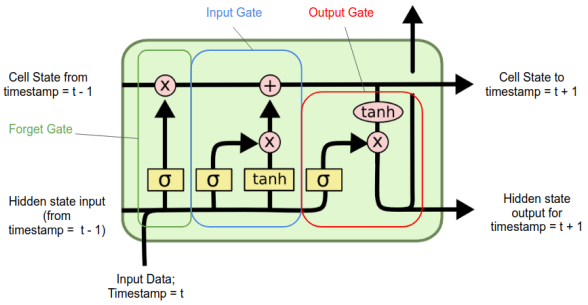

Figure 5: LSTM Cell Structure

GRU (Gated Recurrent Units) → Nived

i. It is a simplified variant of RNN that incorporates gating mechanisms to control the flow of information.

ii. GRUs combine the forget and input gates into a single update gate and remove the cell state, storing both long and short-term memory in the hidden state.

iii. Pros - simpler and faster than more complex models

iv. Cons - Weaker memory, less effective than LSTMs

The GRU equations are:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \tag{7}$$
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \tag{8}$$
$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t]) \tag{9}$$
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \tag{10}$$

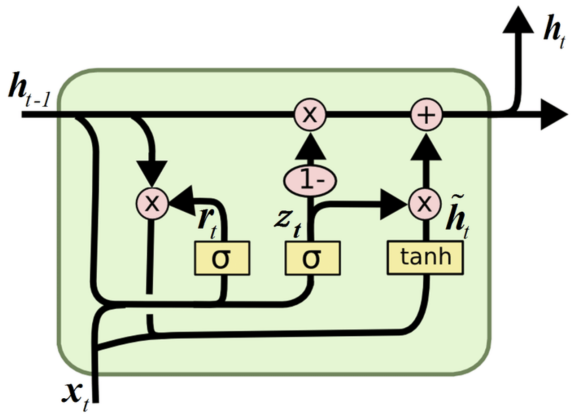

Figure 6: GRU Cell Structure

GRU vs LSTM → Peilin

- **Computational Efficiency:** GRUs are more efficient than LSTMs by merging the forget and input gates into a single *update gate*.
- **Simplified Architecture:** GRUs do not maintain a separate internal cell state; instead, all information is stored directly in the *hidden state*.
- **Speed Advantage:** This simpler structure allows GRUs to train and run faster than LSTMs while still capturing long-term dependencies.

Table 1: Comparison of LSTM and GRU

Feature	**LSTM**	**GRU**
Gates	3 (Input, Forget, Output)	2 (Update, Reset)
Cell State	Yes it has cell state	No (Hidden state only)
Training Speed	Slower due to complexity	Faster due to simpler architecture
Computational Load	Higher due to more gates	Lower due to fewer gates
Performance	Often better in tasks requiring long-term memory	Performs similarly in many tasks with less complexity

CNNs (Convolutional Neural Networks) → Nishant

RNN/LSTM/GRU process the sequence one time step at a time:

$$h_t = f(x_t, h_{t-1}) \tag{11}$$

They are good at long-term memory but do not explicitly model local patterns (e.g., a short price spike followed by reversion) with shared filters.

CNN's were introduced to:

1. Implement weight sharing over time
2. Detect short-term temporal motifs (local shapes) in price and covariate series.
3. Use parallel computation across all time positions.

For a multivariate series with 'd' features:

- At each time t, $x_t \in \mathbb{R}^d$
- A convolutional filter has weights $W \in \mathbb{R}^{k \times d}$.
- The convolution at time t is:

$$h_t = \sum_{k=0}^{K-1} W_k x_{t-k} + b, \tag{12}$$

Where $W_k \in \mathbb{R}^{1 \times d}$ is the slice of the kernel at lag k.
Multiple filters are stacked:

a. Each filter learns a different local pattern (e.g., small uptrend, sharp drop, volatility burst).

b. The layer output is a feature map $H \in \mathbb{R}^{T \times F}$ where F is the number of filters.

Comparison to fully connected layers:

1. Fully-connected (per time step):

$$y_t = \sigma(W_{\text{fc}} x_t + b_{\text{fc}}), \tag{13}$$

uses a different parameter for each input dimension but does not explicitly look at a window of past times.

2. CNN with kernel size K:

$$h_t = \sum_{k=0}^{K-1} W_k x_{t-k} + b \tag{14}$$

uses the same kernel weights W_k across all time positions:

   a. Learns one set of local patterns and reuses it everywhere in the sequence.

   b. Imposes a local stationarity assumption (patterns repeat over time).

How CNN's improve.
RNNs only focus on $\hat{y} = f(h_t)$ with $h_t = f(x_t, h_{t-1})$ which can be slow and does not explicitly emphasise short local shapes.
CNNs replace this with learned local filters:

$$h_t = \sum_{k=0}^{K-1} W_k x_{t-k} + b, \tag{15}$$

This does:

1. Explicitly models short windows of length K.
2. Is fully parallel over t.
3. Learns reusable "templates" for common temporal motifs (e.g., spike-and-revert, mini-trend).

For Use in time series model:
Input:

1. Windowed history of prices, returns, volume, and exogenous features.
2. Example window: $x_{t-K+1}, \ldots, x_t$.

CNN layer(s) produce feature maps capturing:

1. Local trend patterns (short up/down moves).
2. Volatility bursts or calm periods.
3. Interactions between price and covariates (e.g., fuel price + commodity price).

Output of CNN:

1. Flattened or pooled and passed to:
2. A fully-connected layer for prediction, (or) A recurrent layer (LSTM/GRU) in an RCNN architecture (or) An attention mechanism over time.

Strengths:

1. Computational:
    a. Convolutions over time are highly parallelizable on GPUs.
    b. Fixed kernel size K limits the number of parameters.
2. Statistical:
    a. Weight sharing acts as a form of regularization.
    b. Naturally captures local time patterns and short-term dependencies.
3. Practical:
    a. Good fit for high-frequency or dense time series where local structure is dominant.

Limitations:

1. Limited receptive field:
    a. A kernel of size K only sees K steps at a time.
    b. Long-range dependencies require many layers or large K, which can become expensive.
2. Uniform treatment within the window:
    a. Inside the kernel window, each position enters the convolution in a fixed way.
    b. No adaptive "this specific timestep matters more" mechanism.
3. No explicit memory state:

a. Unlike LSTM/GRU, there is no persistent hidden state carrying information far into the future.

b. RCNN: CNN for local features + RNN for long-term memory.

c. Attention: adaptive weighting over time to highlight key positions.

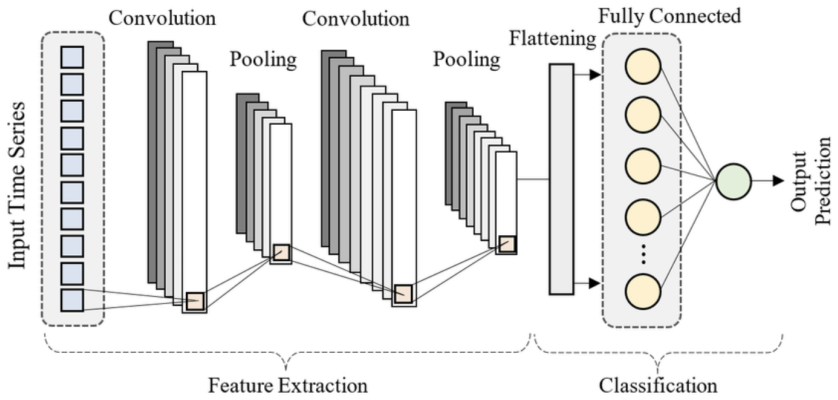

Figure 7: 1D CNN architecture for time series forecasting showing convolutional filters sliding over temporal windows to detect local patterns.

Attention-based RNN/CNN variants → Nishant

i. Improvement from RCNN/CNN:

   (a) Selects the relevant time positions in the conv feature maps.

   (b) Re-weight each time step from the last one adaptively instead of using the previous one.

   (c) Helps to focus on a specific convolution window avoiding information bottlenecks by handling long range dependencies.

ii. Motivation and Plan:

   (a) Not all sequences of data/time steps are important.

   (b) Assign weights to time steps adaptively

iii. Strengths:

   (a) Better handling of long range dependencies

   (b) Focus on event of interest (e.g. shocks, regime changes)

   (c) Adaptively changing weights gives us interpretation of what mattered.

iv. Next steps:

   (a) If the data has more parameters like Prashantha et al. we might require more data and regularization.

   (b) Requires data preprocessing and feature design if we want to handle decomposition and seasonality or trends.

(c) For such longer sequence attention is computationally expensive and a sparse/local attention might be useful.

v. Math:

(a) Score each time step.
h_t = hidden state at time t
W_h, W_q, v: weights and the vector
q = query

$$e_t = \text{score}(h_t, q) \tag{16}$$
$$e_t = v^\top \tanh(W_h h_t + W_q q) \tag{17}$$

(b) Convert scores to weight (Softmax)

$$\alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^{T} \exp(e_j)} \quad \text{for } t = 1 \dots T \text{ and } \sum \alpha_t = 1 \tag{18}$$

These are the attention weights.

(c) Context vector (weighted sum):

$$c = \sum_{t=1}^{T} \alpha_t h_t \tag{19}$$

(d) Prediction from context

$$\hat{y} = f(h_t) \tag{20}$$

(e) To provide context in RNN/LSTM/GRU:
Recurrence:

$$h_t = \text{RNN}(x_t, h_{t-1}) \tag{21}$$

Prediction from last state:

$$\hat{y} = f(h_T) \tag{22}$$

all information must flow through the chain into h_T.
Long range information can be forgotten or compressed too much.
In Attention based models given by equation 3, the model can directly pick earlier states via a larger α_t instead of forcing everything into a single h_T.
In RCNN's it used to follow a mean pooling given by

$$c_{\text{mean}} = \frac{1}{T} \sum_{t=1}^{T} h_t \tag{23}$$

all positions get $\frac{1}{T}$ regardless of importance.
In attention based models "every step counts equally" $\rightarrow$ replaced by **learned, data-dependent weights** that emphasize shocks, regime changes, and important patterns.
In vanilla RNN/LSTM/GRU, the loss at time T backpropagates to earlier steps through a long chain:

$$L(\hat{y}, y) \to h_T \to h_{T-1} \to \cdots \to h_1 \tag{24}$$

This can still cause vanishing gradients and make it hard to learn very long-range dependencies, even with LSTM/GRU.

Gradient flows used in RNN:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}} \tag{25}$$

Gradient flow used in attention based models.

$$c = \sum_t \alpha_t h_t, \quad \hat{y} = f(c), \tag{26}$$

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial c} \cdot \frac{\partial c}{\partial h_t} = \frac{\partial L}{\partial c} \cdot \alpha_t + \text{ terms via } \frac{\partial \alpha_t}{\partial h_t} \tag{27}$$

There is a direct path from loss L to each h_t through the weighted sum, not only through the recurrent chain.

Gradients no longer have to travel exclusively through a long recurrence; attention adds shorter gradient shortcuts to distant time steps.

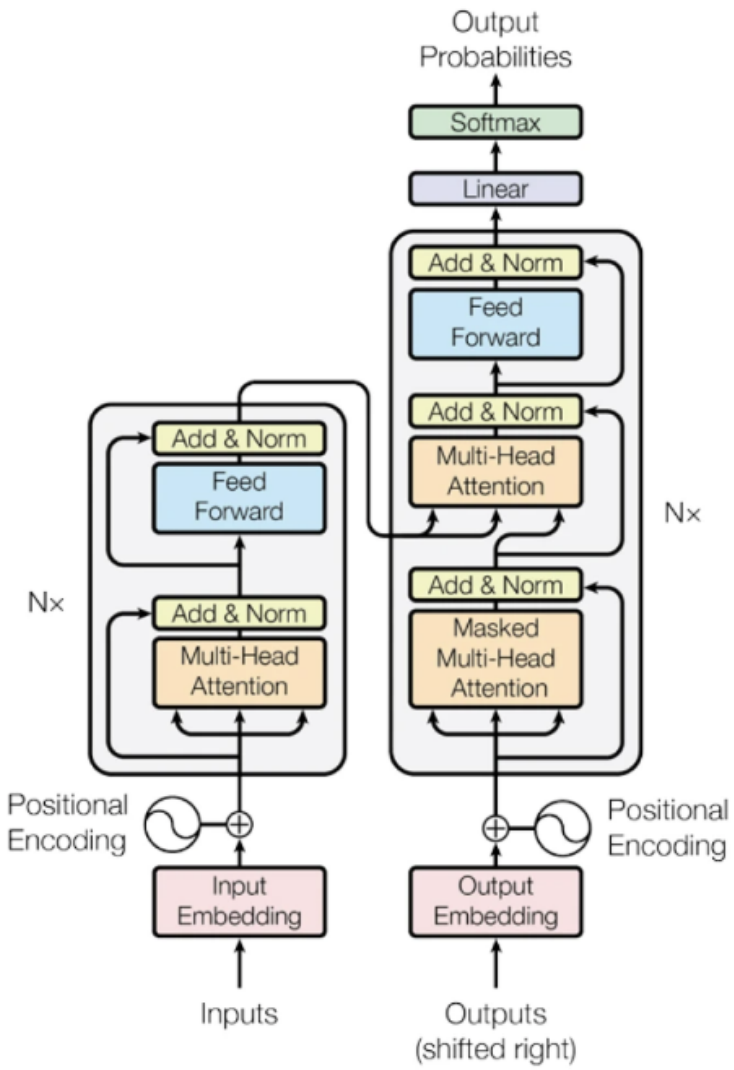

Figure 8: Attention mechanism in time series: the model learns to weight different time steps based on their relevance to the prediction task.

5.1.3 Generative / advanced models

GANs → Nived

Typically outperform LSTMs in some studies; require larger training datasets.

i. GANs or Generative Adversarial Networks consist of two neural networks: a discriminative network (D) that learns to distinguish between real and generated data, and a generative network (G) that aims to produce data indistinguishable from the real data.

ii. The GAN framework drives improvement through competition between the generative model and the discriminative model, where the former attempts to generate realistic data while the latter seeks to detect the generated data as fake.

iii. This adversarial process continues until the generated data becomes indistinguishable from real data.

iv. Pros - Overcomes limited data, acts as good regularization, can be conditioned on specific factors

v. Cons - Instability due to 2 networks, complexity, can oscillate indefinitely without reaching an optimal solution

At Discriminator D:

$$Dloss_{real} = \log(D(x)) \tag{28}$$
$$Dloss_{fake} = \log(1 - D(G(z))) \tag{29}$$
$$Dloss = Dloss_{real} + Dloss_{fake} \tag{30}$$

The total cost is:

$$\frac{1}{m}\sum_{i=1}^{m} \log(D(x^i)) + \log(1 - D(G(z^i))) \tag{31}$$

At Generator G:

$$Gloss = \log(1 - D(G(z))) \text{ or } -\log(D(G(z))) \tag{32}$$

The total cost is:

$$\frac{1}{m}\sum_{i=1}^{m} \log(1 - D(G(z^i))) \tag{33}$$

or

$$\frac{1}{m}\sum_{i=1}^{m} -\log(D(G(z^i))) \tag{34}$$

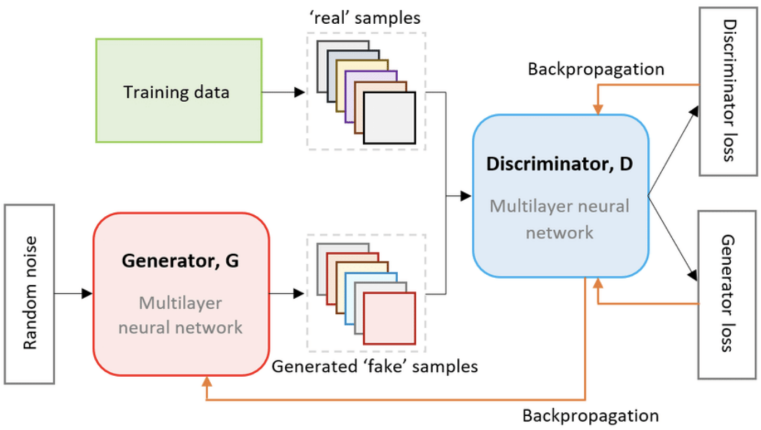

Figure 9: Generative Adversarial Network architecture showing the generator G producing synthetic data and discriminator D distinguishing real from fake samples.

VAE (Variational Autoencoder) → Richard

i. VAE consists of an encoder, a latent space, and a decoder

ii. The encoder consists of layers that gradually become narrower, compressing samples into a lower dimensional latent space (bottleneck) so only useful information is kept (denoising)

iii. Latent space models the mean $\boldsymbol{\mu}$ and variance $\boldsymbol{\sigma}^2$ of each feature (Z_1, Z_2 in the picture below) of its output

iv. The number of features in the output (aka 'latent dimensions', the length of the mean/variance vector, which is 2 in the picture) is defined by the user

v. Random sample vectors taken from latent space distributions ($Z_i = \mu_i + \exp(0.5 * \log(\sigma_i^2)) * \epsilon$, ϵ being a random number taken from the standard normal distribution) are passed to the decoder

vi. Decoder contains layers that gradually become wider, trying to reconstruct the original sample from latent space output

vii. Loss is reconstruction error MSE $= (1/N) * \sum(x_i - \hat{x}_i)^2$ (N is number of sample points, x_i is original sample, $\hat{x}_i$ is reconstructed sample) plus

KL-divergence

$$\sum_j KL(q_j(z|x) \parallel p(z)) \tag{35}$$

$q(z|x)$ is learned latent space distributions, $p(z)$ is target distribution (usually standard normal)

viii. Pros: can serve multiple purposes (dimensionality reduction, denoising, generating synthetic data, anomaly detection)

ix. Cons: the quality of synthetic data generated is usually not as good as that of GANs (no guarantee that random samples from the latent space distributions are close to the latent space outputs of real samples)

Extended Mathematical Formulation: → Nishant

1. Encoder network:

The encoder $q_\phi(z|x)$ maps input x to a distribution over latent space:

$$\mu = f_\mu(x; \phi) \tag{36}$$
$$\log \sigma^2 = f_\sigma(x; \phi) \tag{37}$$

where f_μ and f_σ are neural networks with parameters ϕ.

2. Reparameterization trick:

To enable backpropagation through the stochastic sampling:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \tag{38}$$

3. Decoder network:

The decoder $p_\theta(x|z)$ reconstructs the input from latent samples:

$$\hat{x} = g(z; \theta) \tag{39}$$

4. Evidence Lower Bound (ELBO):

The VAE maximizes the ELBO:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x)\|p(z)) \tag{40}$$

5. KL-divergence (closed form for Gaussians):

$$D_{KL}(q_\phi(z|x)\|p(z)) = -\frac{1}{2}\sum_{j=1}^{J}\left(1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2\right) \tag{41}$$

where J is the latent dimension.

6. Total loss:

$$\mathcal{L}_{\text{total}} = \underbrace{\|x - \hat{x}\|^2}_{\text{Reconstruction}} + \beta \cdot \underbrace{D_{KL}(q_\phi(z|x)\|p(z))}_{\text{Regularization}} \tag{42}$$

where β controls the trade-off (often $\beta = 1$, but β-VAE uses $\beta > 1$ for disentanglement).

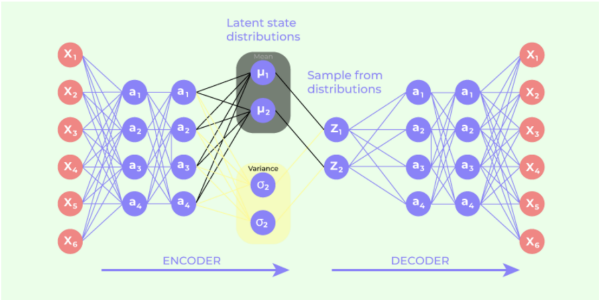

Figure 10: Variational Autoencoder Architecture

Transformers → Richard

i. Encoder and decoder apply embedding (token to vector) and positional encoding on input sequences

ii. Encoder layers consist of multi-head self-attention, feed-forward networks, residual connections and layer normalizations

iii. Decoder layers consist of masked multi-head self-attention (to avoid looking into future information), multi-head cross-attention (between decoder inputs and encoder outputs), residual connections, feedforward networks and layer normalizations

iv. Attention mechanism enables each token to be influenced by and also influence all other tokens in the input sequence by computing tokens' importance scores and then multiplying the scores with the original sequence to get a new representation of the input sequence

v. Multi-head attention lets the model look at information from many different perspectives at once, whereas single-head attention only has one "view." This enables the model to simultaneously attend to different kinds of relationships

vi. Residual connections help solve the vanishing gradient problem by creating a "shortcut" that directly adds the initial input to the layer's output, allowing gradients to bypass some layers during backpropagation

vii. Layer normalization stabilizes and speeds up training by normalizing the inputs to a layer across its features for each individual data point

viii. There are also encoder-only and decoder-only transformer models

ix. Advantage1: Transformers allow the input's different positions to directly interact with each other and can capture long-range dependencies better than RNNs like LSTM

x. Advantage2: Unlike RNNs that must process sequences one token at a time, Transformers process the entire sequence at once, greatly increasing efficiency

xi. Cons: Transformers usually require more data and computing power to train than simpler RNNs

Mathematical Formulation: → Nishant

1. Input embedding and positional encoding:

For input sequence $x = (x_1, \ldots, x_n)$:

$$\text{Input}_i = \text{Embed}(x_i) + \text{PE}(i) \tag{43}$$

Positional encoding using sinusoidal functions:

$$\text{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{44}$$

$$\text{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{45}$$

2. Scaled Dot-Product Attention:

Given queries Q, keys K, and values V:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{46}$$

where d_k is the dimension of keys, and scaling by $\sqrt{d_k}$ prevents dot products from growing too large.

3. Multi-Head Attention:

Parallel attention heads capture different relationship types:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{47}$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{48}$$

with projection matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

4. Feed-Forward Network:

Position-wise FFN applied to each position:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{49}$$

or with GELU activation: $\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$

5. Layer Normalization and Residual Connections:
Each sub-layer output:

$$\text{LayerNorm}(x + \text{Sublayer}(x)) \tag{50}$$

where LayerNorm normalizes across features:

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \tag{51}$$

6. Masked Self-Attention (Decoder):
For autoregressive generation, mask future positions:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V \tag{52}$$

where $M_{ij} = -\infty$ if $j > i$ (masked), else 0.

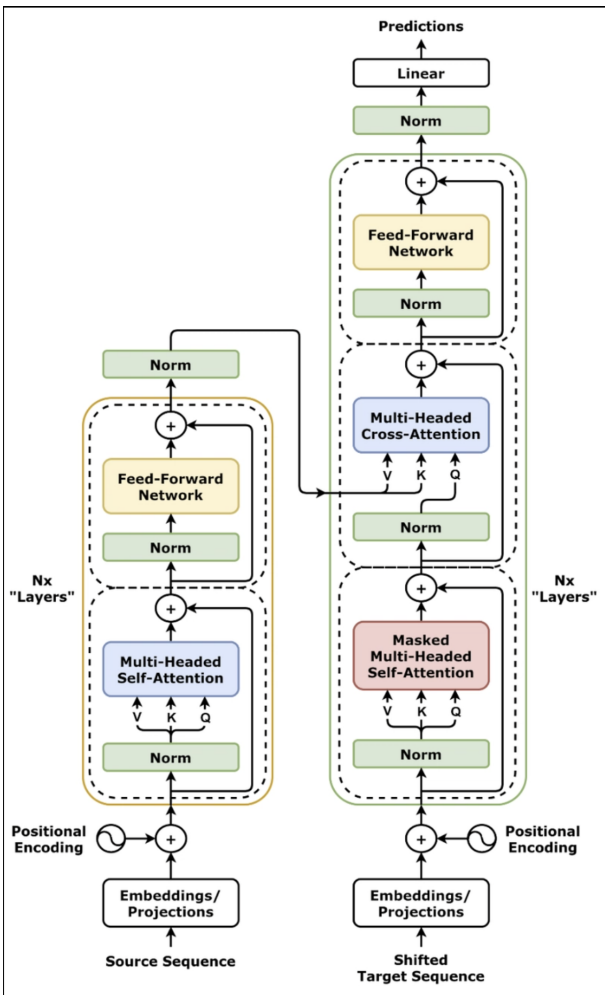

Figure 11: Transformer Architecture

5.1.4 Typical usage patterns

- Price prediction with lagged prices and exogenous inputs.
- Decomposition + DL pipeline (e.g., CEEMDAN + LSTM/TDNN).

5.2 Econometric and time-series models (Richard + Nived)

5.2.1 Standard baselines (Richard)

ARIMA(P,D,Q): autoregressive integrated moving average

$$Y_t = C + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \ldots + \phi_p Y_{t-p} + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \ldots + \theta_q \varepsilon_{t-q} + \varepsilon_t \tag{53}$$

where Y_t: target time series (e.g. price) C: constant ε_t: error
θ/ϕ: coefficients

i. Autoregressive means current value is influenced by P lag values of the time series

ii. Moving average means current value depends on Q lag errors of the model (error measures how much the time series differs from its mean or expected value)

iii. Differencing (D) means calculating the differences between adjacent values in the target time series and fitting the model on these differences as a new series instead of on the original series

iv. P and Q determined by AIC/BIC (Akaike Information Criterion/Bayesian Information Criterion)

v. D determined by ADF test (target series must be stationary)

Seasonal ARIMA (SARIMA(p, d, q, P, D, Q, S))

$$(1 - \phi_1 B)(1 - \Phi_1 B^s)(1 - B)(1 - B^s) y_t = (1 + \theta_1 B)(1 + \Theta_1 B^s)\varepsilon_t \tag{54}$$

B: backshift operator y: target series ε: error

$$\textbf{SARIMA}(\underbrace{1}_{\text{p}}, \underbrace{0}_{\text{d}}, \underbrace{1}_{\text{q}})(\underbrace{2}_{\text{P}}, \underbrace{1}_{\text{D}}, \underbrace{0}_{\text{Q}})\underbrace{12}_{\text{S}}$$

$$y_t - y_{t-12} = W_t \tag{55}$$

$$W_t = \mu + \phi_1 W_{t-1} + \phi_2 W_{t-12} + \phi_3 W_{t-24} + \theta_1 \varepsilon_{t-1} + \varepsilon_t \tag{56}$$

Y_t: target time series (e.g. price) W_t: seasonally differentiated series
ε_t: error θ/ϕ: coefficients μ: constant

5.2.2 Models with exogenous inputs / multivariate structure

SARIMAX: seasonal ARIMA with exogenous variables Mathematical Formulation: → Nishant

SARIMAX extends SARIMA by incorporating exogenous (external) variables X_t:

$$\Phi_P(B^s)\phi_p(B)(1 - B)^d(1 - B^s)^D y_t = \Theta_Q(B^s)\theta_q(B)\varepsilon_t + \sum_{i=1}^{k} \beta_i X_{i,t} \tag{57}$$

where:

- $\phi_p(B) = 1 - \phi_1 B - \phi_2 B^2 - \cdots - \phi_p B^p$ is the non-seasonal AR polynomial

- $\Phi_P(B^s) = 1 - \Phi_1 B^s - \Phi_2 B^{2s} - \cdots - \Phi_P B^{Ps}$ is the seasonal AR polynomial
- $\theta_q(B) = 1 + \theta_1 B + \theta_2 B^2 + \cdots + \theta_q B^q$ is the non-seasonal MA polynomial
- $\Theta_Q(B^s) = 1 + \Theta_1 B^s + \Theta_2 B^{2s} + \cdots + \Theta_Q B^{Qs}$ is the seasonal MA polynomial
- $X_{i,t}$ are exogenous variables (e.g., weather, fuel prices, economic indicators)
- β_i are the regression coefficients for exogenous variables

Key advantage: Allows incorporation of external factors that may influence the target series (e.g., using temperature data to predict agricultural commodity prices).

i. Pros: → Nishant

- Incorporates external predictors (weather, economic indicators) for improved forecasting
- Handles both seasonality and trend simultaneously
- Well-established statistical framework with interpretable coefficients
- Confidence intervals and hypothesis testing available

ii. Cons: → Nishant

- Assumes linear relationship between exogenous variables and target
- Requires stationary data (or appropriate differencing)
- Parameter selection (p, d, q, P, D, Q) can be challenging
- May not capture complex nonlinear dependencies

VAR: vector autoregression for multiple series

$$Y_{1,t} = \alpha_1 + \beta_{1|1} Y_{1,t-1} + \beta_{1|2} Y_{2,t-1} + e_{1,t} \tag{58}$$
$$Y_{2,t} = \alpha_2 + \beta_{2|1} Y_{1,t-1} + \beta_{2|2} Y_{2,t-1} + e_{2,t} \tag{59}$$

Y_1/Y_2: target series 1 and 2 α/β: coefficients e: error
General VAR(p) Formulation: → Nishant
For a k-dimensional time series $\mathbf{Y}_t = (Y_{1,t}, Y_{2,t}, \ldots, Y_{k,t})'$:

$$\mathbf{Y}_t = \mathbf{c} + \mathbf{A}_1 \mathbf{Y}_{t-1} + \mathbf{A}_2 \mathbf{Y}_{t-2} + \cdots + \mathbf{A}_p \mathbf{Y}_{t-p} + \mathbf{e}_t \tag{60}$$

where:

- $\mathbf{c} \in \mathbb{R}^k$ is a vector of constants
- $\mathbf{A}_i \in \mathbb{R}^{k \times k}$ are coefficient matrices for lag i
- $\mathbf{e}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$ is the error vector with covariance matrix $\boldsymbol{\Sigma}$

In compact matrix form:

$$\mathbf{Y}_t = \mathbf{c} + \sum_{i=1}^{p} \mathbf{A}_i \mathbf{Y}_{t-i} + \mathbf{e}_t \tag{61}$$

i. Pros: → Nishant

- Models interdependencies between multiple time series simultaneously
- Enables Granger causality testing to identify lead-lag relationships
- Impulse response analysis shows how shocks propagate through the system
- No need to specify which variables are endogenous vs exogenous

ii. Cons: → Nishant

- Number of parameters grows quadratically with number of series ($k^2 \cdot p$ coefficients)
- Requires all series to be stationary
- Can overfit with too many lags or variables
- Assumes linear relationships between series

5.2.3 Volatility and conditional variance models

GARCH, GJR-GARCH (Richard)

GARCH

$$y_t = x_t' b + \varepsilon_t \tag{62}$$

$$\varepsilon_t | y_{t-1} \sim \mathcal{N}(0, \sigma_t^2) \tag{63}$$

$$\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \cdots + \alpha_q \varepsilon_{t-q}^2 + \beta_1 \sigma_{t-1}^2 + \cdots + \beta_p \sigma_{t-p}^2 = \omega + \sum_{i=1}^{q} \alpha_i \varepsilon_{t-i}^2 + \sum_{i=1}^{p} \beta_i \sigma_{t-i}^2 \tag{64}$$

y: actual return, $x_t b$: predicted return, ε: residual return,
$\varepsilon_t | y_{t-1}$: conditional distribution of residual return given past information,
σ_t^2: conditional variance of residual return given past information,
ω: constant, α/β: coefficients, p/q: number of lags

i. GARCH assumes volatility clustering: periods of high volatility tend to be followed by more high volatility, and similarly for low volatility, which breaks ARIMA's assumption of stationarity

ii. Pros: → Nishant

- Captures volatility clustering observed in financial and commodity markets
- Provides time-varying variance estimates for risk management
- Parsimonious model (few parameters capture complex variance dynamics)
- Well-suited for Value-at-Risk (VaR) calculations

iii. Cons: → Nishant

- Treats positive and negative shocks symmetrically
- Cannot capture leverage effects (asymmetric volatility response)
- Assumes specific distributional form for errors
- May not capture structural breaks in volatility regimes

Stationarity Condition: → Nishant
For GARCH(p,q) to be covariance stationary:

$$\sum_{i=1}^{q} \alpha_i + \sum_{j=1}^{p} \beta_j < 1 \tag{65}$$

The unconditional variance is:

$$\text{Var}(\varepsilon_t) = \frac{\omega}{1 - \sum_{i=1}^{q} \alpha_i - \sum_{j=1}^{p} \beta_j} \tag{66}$$

GJR-GARCH (Richard)

$$\sigma_t^2 = \omega + \sum_{i=1}^{q} (\alpha_i + \gamma_i \cdot I_{t-i}) \cdot \varepsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j \cdot \sigma_{t-j}^2 \tag{67}$$

σ_t^2: Conditional Variance of Residual Return ω: constant
ε_t: residual return $\alpha/\beta/\gamma$: coefficients I: indicator variable (residual return < 0)

i. GJR-GARCH also assumes volatility clustering

ii. GJR-GARCH proposes that positive and negative residual returns may have different levels of effects on volatility (asymmetrical), hence the introduction of the indicator variable I and its coefficient γ

iii. Pros: intuitive, good explainability, relatively fast to fit

iv. Cons: cannot capture non-linear relationships

Leverage Effect Interpretation: → Nishant
The indicator function $I_{t-i} = \mathbf{1}(\varepsilon_{t-i} < 0)$ captures the leverage effect:

- When $\varepsilon_{t-i} < 0$ (negative shock): impact on variance is $(\alpha_i + \gamma_i)\varepsilon_{t-i}^2$
- When $\varepsilon_{t-i} \geq 0$ (positive shock): impact on variance is $\alpha_i \varepsilon_{t-i}^2$

If $\gamma_i > 0$, negative shocks increase volatility more than positive shocks of the same magnitude (common in financial markets).

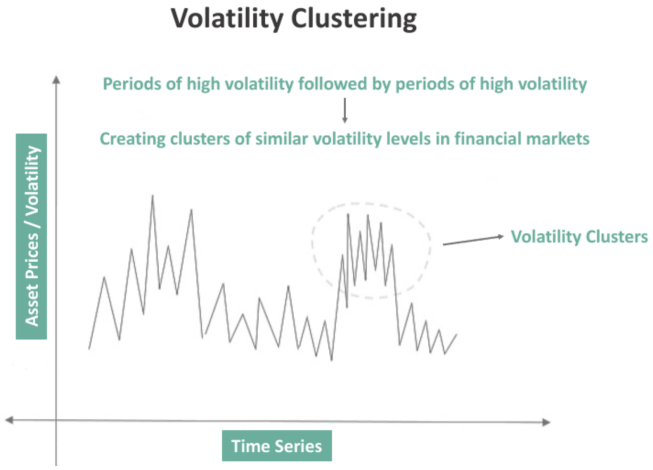

Figure 12: GARCH volatility clustering: periods of high and low volatility tend to persist over time.

MEM-type (Multiplicative Error Models) for volatility / volume (Richard)

$$x_t = \mu_t \varepsilon_t \tag{68}$$

$$\mu_t = \omega + \sum_{i=1}^{q} \alpha_i x_{t-i} + \sum_{j=1}^{p} \beta_j \cdot \mu_{t-j} \tag{69}$$

X: target series (e.g. variance) μ: conditional expectation of x
ε_t: error α/β: coefficients

i. In the MEM model, the target series $X(t)$ must be non-negative, so for volatility forecasting we can set $X(t)$ to be variance. μ is the conditional expectation of $X(t)$ given past information, and μ is multiplied by error ε instead of added, hence the name MEM.

ii. Pros: versatile, can be applied on many different target variables

iii. Cons: cannot capture non-linear relationships, requires target to be non-negative

Error Distribution and Estimation: → Nishant
The error term ε_t is typically assumed to follow a Gamma distribution:

$$\varepsilon_t \sim \text{Gamma}(a, 1/a) \quad \text{with } \mathbb{E}[\varepsilon_t] = 1, \quad \text{Var}(\varepsilon_t) = 1/a \tag{70}$$

This ensures $\mathbb{E}[x_t|\mathcal{F}_{t-1}] = \mu_t$ where $\mathcal{F}_{t-1}$ is the information set up to time $t-1$.
The log-likelihood for MEM with Gamma errors:

$$\ell(\theta) = \sum_{t=1}^{T} \left[(a-1)\log x_t - a \log \mu_t - \frac{a \cdot x_t}{\mu_t} + a \log a - \log \Gamma(a) \right] \tag{71}$$

5.2.4 Decomposition-based approaches

EMD, EEMD, CEEMD, CEEMDAN (Nived)

- Use as preprocessing to split noisy price series into component signals.
- Goal: retain useful structure before feeding into DL or other ML models.

Mathematical Formulation: → Nishant + Nived
Empirical Mode Decomposition (EMD):
EMD decomposes a signal $x(t)$ into Intrinsic Mode Functions (IMFs) and a residue:

$$x(t) = \sum_{i=1}^{n} \text{IMF}_i(t) + r_n(t) \tag{72}$$

where each IMF satisfies: (1) the number of extrema and zero crossings differ by at most one, and (2) the mean of upper and lower envelopes is zero.
Sifting Process:

1. Identify local maxima and minima of $x(t)$
2. Interpolate to form upper envelope $e_{\max}(t)$ and lower envelope $e_{\min}(t)$
3. Compute mean: $m(t) = \frac{e_{\max}(t)+e_{\min}(t)}{2}$
4. Extract detail: $h(t) = x(t) - m(t)$
5. Repeat until $h(t)$ satisfies IMF criteria

Ensemble EMD (EEMD):
To address mode mixing, EEMD adds white noise and averages:

$$\text{IMF}_i^{\text{final}}(t) = \frac{1}{N} \sum_{k=1}^{N} \text{IMF}_i^{(k)}(t) \tag{73}$$

where $\text{IMF}_i^{(k)}$ is the i-th IMF from the k-th noise-added trial.
Complementary EEMD (CEEMD):
Uses pairs of opposite-sign noise to cancel residual noise:

$$\text{IMF}_i^{(\text{CEEMD})}(t) = \frac{1}{2N} \sum_{k=1}^{N} \left(\text{IMF}_i^{(k,+)}(t) + \text{IMF}_i^{(k,-)}(t) \right) \tag{74}$$

where $(k,+)$ and $(k,-)$ denote trials with positive and negative noise respectively.
CEEMDAN (Complete EEMD with Adaptive Noise):
Adds noise at each decomposition stage adaptively:

$$r_i(t) = r_{i-1}(t) - \text{IMF}_i(t), \quad \text{IMF}_{i+1} = \frac{1}{N} \sum_{k=1}^{N} E_1\left(r_i(t) + \epsilon_i w^{(k)}(t)\right) \tag{75}$$

where $E_1(\cdot)$ extracts the first IMF, $w^{(k)}$ is white noise, and ϵ_i controls noise amplitude.

i. Pros: → Nishant

- Data-driven decomposition without predefined basis functions
- Adaptive to local signal characteristics (non-stationary signals)
- Separates different frequency components for targeted modeling
- EEMD/CEEMDAN variants reduce mode mixing artifacts

ii. Cons: → Nishant

- Sensitive to endpoint effects and noise
- Mode mixing can occur in basic EMD
- Ensemble methods (EEMD, CEEMDAN) computationally expensive
- Number of IMFs not known a priori

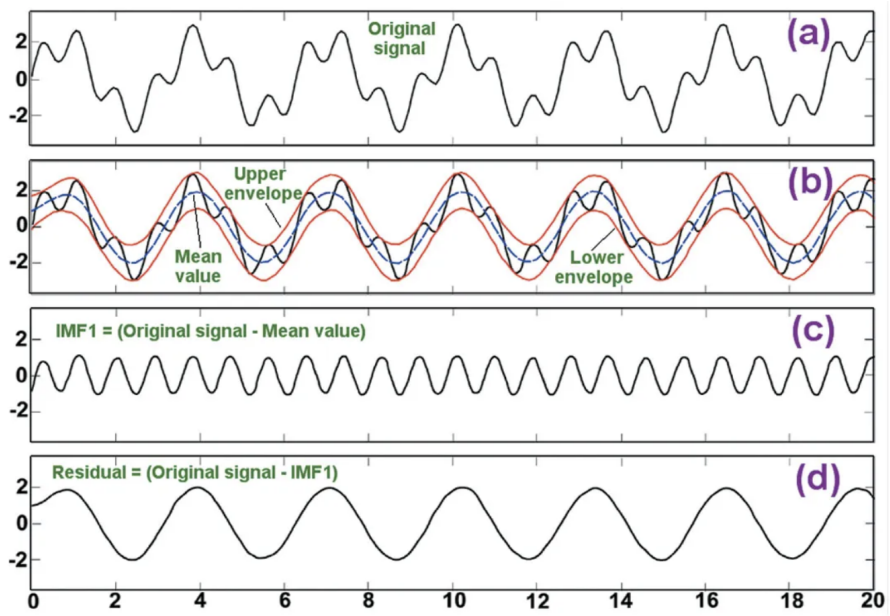

Figure 13: Empirical Mode Decomposition (EMD) sifting process: (a) Original composite signal containing multiple frequency components. (b) Envelope extraction—the upper envelope (red) connects local maxima, the lower envelope (red) connects local minima, and their mean (blue dashed) captures the local trend. (c) First Intrinsic Mode Function (IMF1) obtained by subtracting the mean envelope from the original signal, isolating the highest frequency oscillations. (d) Residual signal after removing IMF1, which becomes the input for extracting subsequent IMFs with progressively lower frequencies.

5.2.5 Comparative notes

- SARIMA / SARIMAX generally outperform plain ARIMA when seasonality and exogenous variables are important.

Model Selection Summary: → Nishant

Table 2: Comparison of Econometric Time Series Models

Model	Best For	Handles	Limitation
ARIMA	Stationary series	Trend, autocorrelation	No seasonality
SARIMA	Seasonal patterns	Seasonality + trend	No external factors
SARIMAX	External influences	Exogenous variables	Linear relationships
VAR	Multiple series	Cross-correlations	Dimensionality
GARCH	Volatility clustering	Heteroskedasticity	Symmetric shocks
GJR-GARCH	Leverage effects	Asymmetric volatility	Still parametric
MEM	Non-negative series	Volume, variance	Requires $x_t \geq 0$

5.3 Tree-based and ensemble models (Richard + Nived)

5.3.1 Use in our applied papers

- Prashantha et al.: list decision tree and random forest as candidate models.
- Shanthini et al.: train decision tree, random forest, XGBoost, LightGBM and CatBoost on onion + weather data.

5.3.2 Models commonly reported in the 3 review papers

Decision Trees (Richard)

i. Each sample starts from the root node

ii. For each split of the current node the tree picks a feature from the dataset and ask a yes/no question about that sample (e.g. is sample's feature value > a specific threshold) so the sample goes down to one of the two child nodes

iii. Each child node can be split again; if it's not split (no child nodes of its own) then it becomes a leaf node

iv. All samples will end up on one of the leaf nodes

v. The average of the samples on each leaf node becomes that leaf node's prediction (for regressor trees)

vi. Decision trees are weak learners: they perform slightly better than random guessing

vii. Pros: → Nishant

- Highly interpretable—decision rules can be visualized and explained
- No feature scaling or normalization required
- Handles both numerical and categorical data
- Fast inference time

viii. Cons: → Nishant

- Prone to overfitting, especially with deep trees
- High variance—small changes in data can produce very different trees

- Greedy splitting may miss globally optimal structure
- Poor extrapolation beyond training data range

Mathematical Formulation: → Nishant
Splitting Criterion:
For a node m with samples Q_m, find the best split (j, t_m) on feature j at threshold t_m:

$$(j^*, t_m^*) = \arg\min_{j,t_m} \left[\frac{|Q_m^L|}{|Q_m|} H(Q_m^L) + \frac{|Q_m^R|}{|Q_m|} H(Q_m^R) \right] \tag{76}$$

where $Q_m^L = \{(x, y) \in Q_m : x_j \leq t_m\}$ and $Q_m^R = Q_m \setminus Q_m^L$.
For Regression (MSE):

$$H(Q_m) = \frac{1}{|Q_m|} \sum_{(x,y)\in Q_m} (y - \bar{y}_m)^2, \quad \bar{y}_m = \frac{1}{|Q_m|} \sum_{(x,y)\in Q_m} y \tag{77}$$

Prediction at leaf node m:

$$\hat{y}_m = \bar{y}_m = \frac{1}{|Q_m|} \sum_{(x,y)\in Q_m} y \tag{78}$$

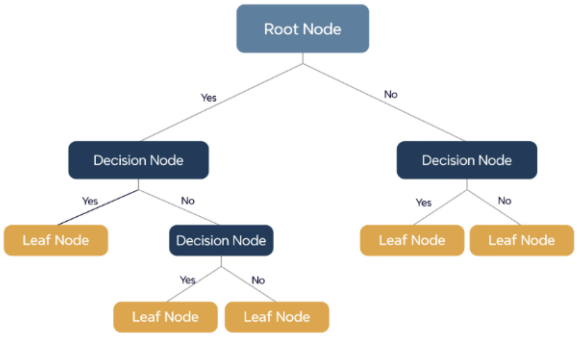

Figure 14: Decision Tree Structure

Random Forest (Nived)

i. Pros - Robust to overfitting, handles outliers/missing data, feature importance scores

ii. Cons - May not capture complex relationships as effectively as the rest

Mathematical Formulation: → Nishant
Bagging (Bootstrap Aggregating):
Random Forest trains B trees on bootstrap samples. For each tree b:

1. Draw n samples with replacement from training data
2. At each split, consider only $m \approx \sqrt{p}$ random features (for classification) or $m \approx p/3$ (for regression)
3. Grow tree T_b without pruning

Ensemble Prediction (Regression):

$$\hat{y} = \frac{1}{B} \sum_{b=1}^{B} T_b(x) \tag{79}$$

Out-of-Bag (OOB) Error:

For each observation i, predict using only trees where i was not in the bootstrap sample:

$$\hat{y}_i^{\text{OOB}} = \frac{1}{|S_i|} \sum_{b \in S_i} T_b(x_i), \quad S_i = \{b : i \notin \text{bootstrap sample } b\} \tag{80}$$

Feature Importance (Permutation):

$$\text{Importance}(j) = \frac{1}{B} \sum_{b=1}^{B} \left[\text{OOB}_b^{\text{permuted}(j)} - \text{OOB}_b \right] \tag{81}$$

Table 3: Impurity Measures for Decision Trees

Impurity	**Task**	**Formula**	**Description**
Gini impurity	Classification	$\sum_{i=1}^{C} f_i(1 - f_i)$	f_i is the frequency of label i at a node and C is the number of unique labels.
Entropy	Classification	$\sum_{i=1}^{C} -f_i \log(f_i)$	f_i is the frequency of label i at a node and C is the number of unique labels.
Variance / Mean Square Error (MSE)	Regression	$\frac{1}{N} \sum_{i=1}^{N} (y_i - \mu)^2$	y_i is label for an instance, N is the number of instances and μ is the mean given by $\frac{1}{N} \sum_{i=1}^{N} y_i$
Variance / Mean Absolute Error (MAE) (Scikit-learn only)	Regression	$\frac{1}{N} \sum_{i=1}^{N} \lvert y_i - \mu \rvert$	y_i is label for an instance, N is the number of instances and μ is the mean given by $\frac{1}{N} \sum_{i=1}^{N} y_i$

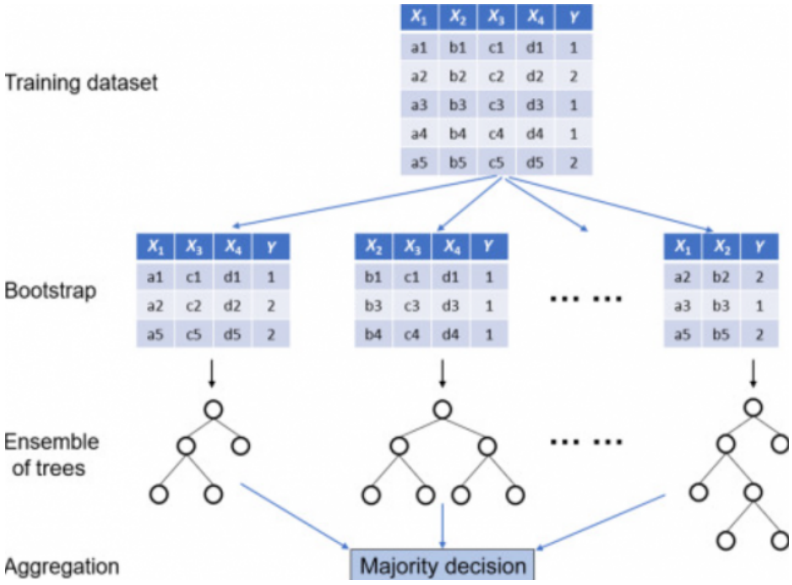

Figure 15: Random Forest architecture: the training dataset is sampled with replacement (bootstrap) to create multiple subsets, each containing a random selection of features. Independent decision trees are trained on each bootstrap sample, and their predictions are aggregated through majority voting (classification) or averaging (regression) to produce the final output.

Gradient Boosting (Nived)

i. Gradient boosting is a powerful machine learning ensemble technique used for regression and classification tasks, known for its predictive accuracy.

ii. It builds models in a stage-wise fashion and generalizes them by optimizing an arbitrary differentiable loss function.

iii. Pros - Handles mixed data, incorporates weak learners efficiently

iv. Cons - prone to overfitting, slower and not scalable for large datasets

$$\gamma_j = \frac{\sum_{x_i \in R_j} (y_i - p)}{\sum_{x_i \in R_j} p(1-p)}$$

γ is computed for each terminal node j

Aggregating for all the data points x_i that belongs to terminal node j

Figure 16: Gradient Boosting Process

XGBoost (Richard)

i. XGBoost is an ensemble model made up of many decision trees

ii. Boosting means trees are grown sequentially: for each new tree added, the model calculates the gradient of the loss function with respect to all previous trees' combined prediction and trains the new tree to correct the error of all previous trees' combined prediction

iii. XGBoost has built in L1/L2 regularization, tree-pruning and missing value handling.

iv. Pros: → Nishant

- State-of-the-art accuracy on structured/tabular data
- Built-in L1/L2 regularization prevents overfitting
- Handles missing values automatically
- Parallel processing for faster training
- Feature importance and model interpretability tools

v. Cons: → Nishant

- Many hyperparameters require careful tuning
- Can overfit on small or noisy datasets
- Memory intensive for very large datasets
- Sequential tree building limits parallelization benefits

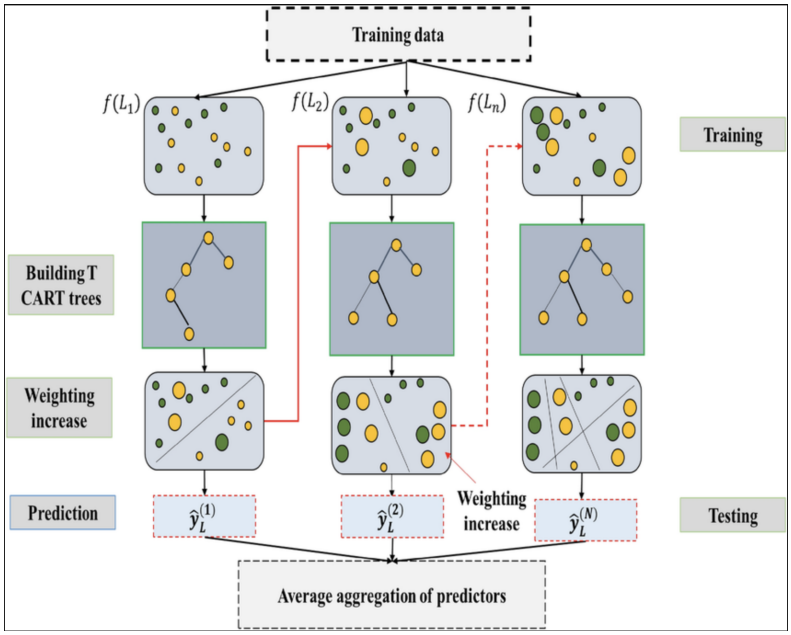

Figure 17: XGBoost architecture: training data is processed through sequential CART trees, where each iteration increases weights on previously misclassified samples (shown as larger circles). Each tree produces a prediction $\hat{y}_L^{(i)}$, and the final output is obtained by aggregating all individual predictors.

LightGBM (Richard)

i. Similar gradient boosting mechanisms as in XGBoost

ii. LightGBM has a leaf-wise growth strategy: it always splits the leaf that will bring the maximum gain (reduction in loss) no matter where that leaf is, resulting in asymmetrical trees

iii. LightGBM buckets the continuous feature values into a relatively small number of discrete bins

iv. LightGBM only considers the boundaries between the bins for splitting (histogram split)

v. XGBoost now supports histogram split too

vi. LightGBM now supports "linear trees": instead of predicting the average of all samples on each leaf, a linear regression model is trained for each leaf's samples and used to make predictions

vii. LightGBM is optimized for large datasets

viii. Pros: → Nishant

- Significantly faster training than XGBoost on large datasets
- Lower memory usage due to histogram-based splitting
- Handles large-scale data efficiently (millions of samples)
- Supports categorical features directly without encoding

ix. Cons: → Nishant

- Leaf-wise growth can overfit on small datasets
- Sensitive to hyperparameters (especially `num_leaves`)
- May produce less balanced trees than level-wise methods
- Requires careful tuning of `min_data_in_leaf` to prevent overfitting

CatBoost (Richard)

i. Gradient boosting in CatBoost is similar to XGBoost/LightGBM's cases

ii. "symmetric" / "oblivious" tree structure: at each tree depth level, all splits happen using the same feature and threshold condition

iii. CatBoost computes special numerical transformations (CTRs) of categorical feature values while using a technique (ordered permutations) to avoid target leakage

iv. CatBoost is optimized for datasets with many categorical features

v. XGBoost and LightGBM now both support categorical features without need for manual preprocessing, but their methods may not be as mature/effective as Catboost's

vi. Pros: → Nishant

- Best-in-class handling of categorical features (no manual encoding needed)
- Ordered boosting reduces prediction shift and overfitting
- Symmetric trees enable fast inference
- Robust default hyperparameters—works well out-of-the-box

vii. Cons: → Nishant

- Slower training than LightGBM on purely numerical data
- Symmetric tree constraint may limit flexibility
- Higher memory consumption during training
- Less community support compared to XGBoost

5.3.3 Strengths (Richard)

- Handle mixed data types (continuous + categorical) well
- Native support for missing values (XGBoost, LightGBM, CatBoost)
- Capture nonlinearities and interactions between features
- Provide feature importance (RF, XGBoost, LightGBM, CatBoost)
- LightGBM: usually fastest on large-scale data
- CatBoost: usually the best performer when there are a lot of categorical features and the categorical features have high cardinality (many unique categories)
- Random Forest: robust to overfitting, good default choice, stable performance

5.3.4 Weaknesses / caveats (Richard)

- CatBoost is generally slower than XGBoost/LightGBM, especially for very large datasets
- Poor extrapolation ability: tree based models are not good at making predictions outside of training sample's target ranges (as compared to methods like linear regression)
- Vanilla gradient boosting (GBDT) is prone to overfitting
- XGBoost / LightGBM/CatBoost: have built-in regularization, but need some tuning efforts

Computational Complexity Comparison: → Nishant

Table 4: Complexity and Characteristics of Tree-Based Methods

Method	**Training**	**Inference**	**Tree Growth**	**Best For**
Decision Tree	$O(n \cdot d \cdot \log n)$	$O(\log n)$	Single tree	Interpretability
Random Forest	$O(B \cdot n \cdot m \cdot \log n)$	$O(B \cdot \log n)$	Parallel	Stability
XGBoost	$O(M \cdot n \cdot d)$	$O(M \cdot \log n)$	Level-wise	Accuracy
LightGBM	$O(M \cdot k \cdot d)$	$O(M \cdot D)$	Leaf-wise	Large data
CatBoost	$O(M \cdot n \cdot d)$	$O(M \cdot D)$	Oblivious	Categorical

n: samples, d: features, B: trees (RF), M: boosting rounds, m: feature subset, k: histogram bins, D: tree depth

5.4 Other machine-learning models (Peter)

5.4.1 k-Nearest Neighbors (k-NN)

k-NN is an instance-based or "lazy learning" method. It does not learn a model from the data; it simply stores the entire training dataset. All the computation happens at prediction time.

i. Distance Calculation:

When you provide a new, unseen query point (x_q), the algorithm calculates the distance between it and every point (x_i) in the training set. The most common metric is Euclidean distance:

$$d(x_q, x_i) = \sqrt{\sum_j (x_{qj} - x_{ij})^2} \tag{82}$$

Note: Because it is distance-based, feature scaling (e.g., standardizing all features) is essential.

ii. Prediction:

The algorithm finds the "k" (a number you choose) training points that are closest to the new point.

- For regression: The prediction is the average of the target values of those k neighbors.
- For classification: The prediction is the majority vote (mode) of the classes of those k neighbors.

iii. Pros:

- Very simple to understand and implement.
- Non-parametric: makes no assumptions about the data distribution.
- Works for both classification and regression.

iv. Cons:

- Computationally slow at prediction time because it must compare to all N training points.
- Suffers from the "curse of dimensionality"; performance drops as the number of features increases.
- Sensitive to the scale of features and to irrelevant features.

5.4.2 Linear Regression

i. This is a parametric model that assumes a linear relationship between the features (x) and a continuous target (y).

ii. The model is a linear equation:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p \tag{83}$$

where $\hat{y}$ is the predicted value, β_0 is the intercept, and $\beta_1 \ldots \beta_p$ are the feature coefficients.

iii. The Training (Ordinary Least Squares):

The model finds the β values that minimize the Sum of Squared Errors (SSE):

$$\text{Cost} = \sum_i (y_i - \hat{y}_i)^2 \tag{84}$$

5.4.3 Bayesian Regression

i. This is a probabilistic version of linear regression. Instead of finding a single best value for each coefficient β, it finds an entire probability distribution.

ii. It uses Bayes' theorem to update prior beliefs about the coefficients with the likelihood of the observed data:

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior} \tag{85}$$

iii. The output is not a single value like "$\beta_1 = 5.2$." Instead, it gives a distribution, such as "there is a 95% probability that β_1 is between 4.9 and 5.5." This allows uncertainty quantification.

iv. Pros:

- Linear (OLS): very fast, simple, interpretable, and a strong baseline.
- Bayesian: provides uncertainty estimates.
- Bayesian: works well on small datasets by using prior beliefs.

v. Cons:

- Linear: assumes linearity and constant error variance.
- Linear: sensitive to outliers.
- Linear: cannot capture complex non-linear relationships.
- Bayesian: more computationally expensive.

5.4.4 Support Vector Machines (SVM) & Support Vector Regression (SVR)

i. The goal of SVM is to find the optimal hyperplane that separates two classes while maximizing the margin (the "empty street") between the closest points of each class. These closest points are called Support Vectors.

ii. SVR applies similar logic to regression. Instead of maximizing separation between classes, it fits an "epsilon-insensitive tube" around the regression function.

- Points inside the tube are not penalized (zero error).

- Points outside the tube are penalized, making SVR robust to outliers.

iii. If the data is not linearly separable, the kernel trick projects the data into a higher-dimensional space where it becomes separable — without explicitly computing the transformation.

iv. A kernel function $K(x_i, x_j)$ computes the relationship between two points as if they were in the transformed feature space.

v. Common kernels:

- Linear
- Polynomial
- RBF (Radial Basis Function), the most common for complex non-linear data

vi. Pros:

- Very effective in high-dimensional spaces.
- Kernel trick allows modeling complex, non-linear boundaries.
- SVR is robust to outliers because of the epsilon-insensitive tube.
- Memory efficient — only uses support vectors.

vii. Cons:

- Can be slow to train on large datasets (complexity can be quadratic or cubic in the number of samples).
- Often behaves as a "black box" with low interpretability.
- Performance depends heavily on hyperparameter tuning (kernel, C, gamma).

5.5 Hybrid and decomposition-based models (Nived+Richard)

5.5.1 Decomposition + model frameworks

(from Theofilou et al. and others)

- Decompose original price series into components, then model each or recombined series.
- Common decomposer: CEEMDAN.

5.5.2 Examples of hybrid architectures

CEEMDAN–TDNN:

- CEEMDAN decomposes the series into intrinsic mode functions.
- TDNN (time-delay neural network) models the decomposed components.

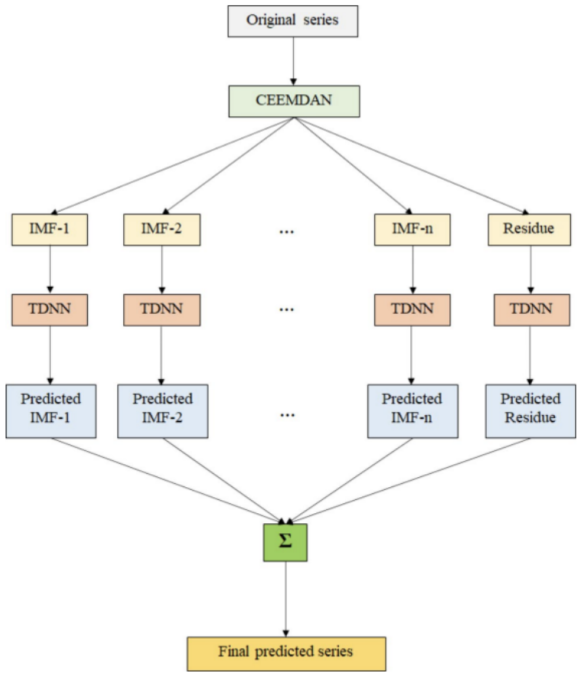

Figure 18: CEEMDAN-TDNN Hybrid Architecture

LSTM–VAE / BiLSTM–VAE: (Richard)

- Replace MLP/FFN layers in the encoder and decoder of a VAE with (Bi)LSTM layers for sequence modelling
- Enables VAE to capture temporal dependence in time-series input data

LSTM–SARIMA: (Richard)

- LSTM better at capturing complex temporal dependence but prone to overfitting, SARIMA more stable and explainable
- Combine LSTM and SARIMA predictions via weighted average
- weights proportional to LSTM / SARIMA's individual performance in out of sample data (inversely proportional to loss/error)

LSTM–GARCH: (Richard)

- Fit GARCH to residual returns from LSTM forecasts to estimate conditional variance
- Residual returns = actual return on day t - LSTM predicted return for day t

5.5.3 Decomposition and multi-stage hybrid pipelines (Nishant)

- STL–LOESS decomposition for yield forecasting:
  - Use seasonal-trend decomposition (STL) with LOESS smoothing to separate trend/seasonality effects before forecasting yield.

  - Compare multiple time-series baselines for yield (e.g., AR/MA/ARMA/ARIMA/ES variants), selecting the best performer by forecast error.

- Sequential (multi-stage) hybrid pipeline: yield → supply/demand → price
  - Forecast crop yield first; then construct supply as a derived variable (e.g., predicted yield + residue + imports).
  - Predict demand using the most informative covariates (in some cases, year/timestamp alone).
  - Predict final crop price using supply, demand, and time with a downstream predictor (statistical regression and/or ML models).

STL Decomposition Mathematical Formulation: → Nishant
STL decomposes a time series Y_t into three additive components:

$$Y_t = T_t + S_t + R_t \tag{86}$$

where:

- T_t is the trend component (long-term movement)
- S_t is the seasonal component (periodic patterns)
- R_t is the remainder/residual component

LOESS (Locally Estimated Scatterplot Smoothing) fits local weighted polynomial regressions:

$$\hat{T}_t = \sum_{i=1}^{n} w_i(t) Y_i, \quad w_i(t) = K\left(\frac{|t-i|}{h}\right) \tag{87}$$

where $K(\cdot)$ is a kernel function (typically tricube) and h is the bandwidth parameter.

i. Pros: → Nishant
   - Robust to outliers in the seasonal and trend components
   - Handles any type of seasonality (not limited to fixed periods)
   - Allows seasonal component to evolve over time
   - Well-suited for agricultural data with varying seasonal patterns

ii. Cons: → Nishant
   - Computationally more expensive than classical decomposition
   - Requires tuning of smoothing parameters
   - Assumes additive decomposition (multiplicative requires log transformation)
   - May struggle with abrupt structural changes

5.5.4 Ensemble-style hybrid predictors used in our papers (Nishant)

- Bagging-based ensembles:
  - Random Forest (RF) used as a strong nonlinear baseline for commodity/vegetable price prediction.
- Boosting-based ensembles:
  - Gradient Boosting Machine (GBM) used for price forecasting across multiple markets.
  - XGBoost-style boosted tree models appear as additional boosted ensemble baselines in our set.
- Hybrid benchmarking "suites" (ensemble in the broader sense of model families):
  - Some of our papers evaluate (side-by-side) classical time-series models together with ML predictors (e.g., ARIMA family vs SVR/RF/GBM/NN variants) to identify robust performers across regimes and markets.

Ensemble Combination Strategies: → Nishant + Richard
When combining multiple model predictions, common strategies include:
1. Simple Average:

$$\hat{y}_{\text{ensemble}} = \frac{1}{M} \sum_{m=1}^{M} \hat{y}_m \tag{88}$$

2. Weighted Average (based on validation performance):

$$\hat{y}_{\text{ensemble}} = \sum_{m=1}^{M} w_m \hat{y}_m, \quad \text{where } \sum_{m=1}^{M} w_m = 1 \tag{89}$$

Weights can be determined by inverse validation error:

$$w_m = \frac{1/\text{MSE}_m}{\sum_{j=1}^{M} 1/\text{MSE}_j} \tag{90}$$

3. Stacking (Meta-learner):

$$\hat{y}_{\text{final}} = g(\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_M; \theta) \tag{91}$$

where $g(\cdot)$ is a meta-model (e.g., linear regression, neural network) trained on base model predictions.

i. Pros: → Nishant

- Reduces variance through averaging diverse model predictions
- Often outperforms individual models, especially when models have uncorrelated errors
- Provides robustness across different market regimes
- Weighted ensembles can adapt to model strengths

ii. Cons: → Nishant

- Increased computational cost (training multiple models)
- Weight optimization may overfit on validation data
- Reduced interpretability compared to single models
- Diminishing returns if base models are highly correlated

4. Competitive Ensemble (Richard)

- still experimental, no widespread use
- All base models trained independently
- For test set samples, use methods like KNN to find several validation set samples that are closest to it
- Use the base model that performed the best on the validation samples to make the prediction for the test sample

5.5.5 Summary of Hybrid Approaches

→ Nishant

Table 5: Comparison of Hybrid Model Architectures

Hybrid Type	**Components**	**Strength**	**Use Case**
CEEMDAN–TDNN	Decomposition + NN	Handles non-stationarity	Noisy price series
LSTM–VAE	Sequence + Generative	Uncertainty quantification	Synthetic data, anomaly
LSTM–SARIMA	DL + Statistical	Combines strengths	Seasonal + complex patt
LSTM–GARCH	Mean + Variance model	Volatility forecasting	Risk management
STL–ARIMA	Decomposition + TS	Trend/season separation	Agricultural yields
Multi-stage	Sequential pipeline	Domain structure	Supply-demand-price
Ensemble suite	Model averaging	Robustness	Cross-market forecasting

5.5.6 General motivation

- Use decomposition to isolate structure / noise.
- Combine strengths of DL, econometric and ensemble models in a single pipeline.

6 Underlying Model Assumptions

(Nishant)

6.1 Data availability and quality

6.1.1 Regular, good-quality market data

- Price quotes are recorded consistently over time.
- Missing data, outliers and reporting errors are limited or can be cleaned.
- Near-term prices can be predicted from past prices and basic market identifiers (assumed in Prashantha et al.).

6.1.2 Stable market definitions

- States, districts and markets are well-defined and comparable over time.
- Commodity and variety labels are consistent across the sample.

6.1.3 Local weather as a key driver

- Weather variables are available for the same area and date as the target price (assumed in Shanthini et al.).
- Short-run weather shocks (temperature, rainfall, humidity, etc.) materially affect supply and prices.

6.2 Statistical properties in classical time-series models

6.2.1 Stationarity (or transformable to stationarity)

- The underlying series (or its differences / returns) has stable mean and variance over time.
- No persistent trends or structural breaks after appropriate differencing.

6.2.2 Linear autoregressive structure

- Future values can be expressed as linear combinations of past values (and possibly past shocks).
- Error terms are uncorrelated, zero-mean and homoscedastic (before adding volatility models).

6.2.3 Seasonality and exogenous inputs

- SARIMA / SARIMAX assume regular, repeatable seasonal patterns.
- When exogenous variables are used, they are measured without large errors and enter linearly (or via simple transformations).

6.3 Assumptions in deep learning and hybrid models

6.3.1 Rich covariate availability

- Weather, production, yield and sometimes macro / trade indicators are available alongside prices.
- These covariates capture meaningful variation in supply, demand and costs.

6.3.2 Temporal and spatial alignment

- Prices and exogenous variables are aligned on the same time grid.
- When spatial information is used, regions are comparable and mapping is stable.

6.3.3 Data volume and representativeness

- Enough historical data exist to train complex models (LSTMs, GANs, Transformers, hybrids).
- Training data are broadly representative of the regimes in which the model will be used.

6.3.4 Stability of relationships

- The relationship between prices and covariates does not change too abruptly over time.
- Hybrid models (e.g., decomposition + DL) assume that decomposed components remain interpretable and predictive.

7 Evaluation Protocols and Metrics

7.1 Point forecast error metrics (Nived)

7.1.1 Common Error Metrics

Root Mean Squared Error (RMSE)

- Most common primary metric when the downstream task is price prediction.
- Penalizes larger errors more heavily.

Mathematical Formulation: → Nishant

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{92}$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and n is the number of observations.

Interpretation: → Nishant

- **Low RMSE:** Model predictions are close to actual values; good predictive accuracy.

- **High RMSE:** Large deviations between predictions and actuals; poor model fit.
- RMSE is in the same units as the target variable, making it directly interpretable.
- Due to squaring, RMSE is more sensitive to outliers/large errors than MAE.

Mean Absolute Error (MAE) **Mathematical Formulation:** → Nishant

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{93}$$

Interpretation: → Nishant

- **Low MAE:** Predictions are consistently close to actual values on average.
- **High MAE:** Model has large average prediction errors.
- MAE treats all errors equally (linear penalty), unlike RMSE's quadratic penalty.
- More robust to outliers than RMSE; preferred when outliers should not dominate evaluation.

Mean Absolute Percentage Error (MAPE) **Mathematical Formulation:** → Nishant

$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{94}$$

Interpretation: → Nishant

- **Low MAPE:** Small percentage errors; model captures the scale of values well.
- **High MAPE:** Large relative errors; predictions deviate significantly as a proportion of actual values.
- Scale-independent, allowing comparison across different commodities or price ranges.
- **Caution:** Undefined or inflated when $y_i \approx 0$; asymmetric (penalizes under-predictions more than over-predictions).

Mean Absolute Scaled Error (MASE) **Mathematical Formulation:** → Nishant

$$\text{MASE} = \frac{\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|}{\frac{1}{n-1}\sum_{i=2}^{n}|y_i - y_{i-1}|} \tag{95}$$

The denominator is the MAE of a naïve random walk forecast (predicting $\hat{y}_i = y_{i-1}$).
Interpretation: → Nishant

- **MASE** < 1**:** Model outperforms the naïve baseline; predictions add value.
- **MASE** $= 1$**:** Model performs equally to naïve forecast.
- **MASE** > 1**:** Model is worse than simply predicting the previous value.
- Scale-independent and symmetric; well-suited for comparing across different time series.
- Recommended for intermittent demand or series with zeros (avoids MAPE's division issues).

Relative Root Mean Squared Error (RRMSE) **Mathematical Formulation:** → Nishant

$$\text{RRMSE} = \frac{\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}}{\bar{y}} = \frac{\text{RMSE}}{\bar{y}} \tag{96}$$

where $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$ is the mean of actual values.
Interpretation: → Nishant

- **Low RRMSE:** Errors are small relative to the average magnitude of the target.
- **High RRMSE:** Errors are large compared to typical values in the series.
- Expressed as a proportion/percentage; enables cross-commodity comparison.
- Useful when comparing models across datasets with different scales.

7.1.2 Metric Selection Guide

→ Nishant

Table 6: Summary of Point Forecast Error Metrics

Metric	Scale	Outlier Sensitivity	Interpretability	Best Use Case
RMSE	Same as y	High (quadratic)	Direct (units)	When large errors are costly
MAE	Same as y	Low (linear)	Direct (units)	Robust general evaluation
MAPE	Percentage	Medium	Intuitive (%)	Cross-scale comparison
MASE	Dimensionless	Low	Relative to naïve	Series with zeros/intermittent
RRMSE	Dimensionless	High	Relative to mean	Cross-dataset comparison

Practical Recommendations: → Nishant

- Use **RMSE** when large forecast errors have disproportionate consequences (e.g., inventory planning).
- Use **MAE** for a balanced view of average error magnitude, especially with outliers.
- Use **MAPE** for business communication (intuitive percentage interpretation), but avoid if y_i can be near zero.
- Use **MASE** for rigorous model comparison against a naïve baseline, especially in academic benchmarks.
- Use **RRMSE** when comparing forecast accuracy across commodities with different price levels.

Relationship Between Metrics: → Nishant
For any dataset, the following inequality always holds:

$$\text{MAE} \leq \text{RMSE} \leq \sqrt{n} \cdot \text{MAE} \tag{97}$$

The gap between RMSE and MAE indicates error variance: if RMSE $\approx$ MAE, errors are uniform; if RMSE $\gg$ MAE, a few large errors dominate.

7.2 Using multiple metrics together

7.2.1 Complementary views of performance

- Combine RMSE with MAE / MAPE to capture both scale and relative error.
- Use MASE / RRMSE to compare across series with different scales.

7.2.2 Strategy-level and economic evaluation

7.2.3 Trading / investment strategies based on forecasts

- Long-only strategies (e.g., buy when predicted price rises).
- Long–short or hedging strategies using predicted price movements.

7.3 Performance metrics for these strategies

- Cumulative return.
- Maximum drawdown.
- Sharpe ratio (risk-adjusted return).

8 Representative Datasets and Task Map

8.1 Planned core dataset: Vietnam coffee + fuel + weather

8.1.1 Data fields (Nived)

- Coffee spot price (daily, Vietnam).
- Fuel price series (e.g., diesel, gasoline).
- Local weather: temperature, humidity, rainfall (plus optional wind, pressure, etc.).

8.1.2 Primary tasks

- One-step-ahead coffee price prediction.
- Short multi-step price forecasts (few days / weeks ahead).

8.1.3 Secondary tasks (Nived)

- Feature importance analysis (fuel vs weather vs past price).
- Scenario analysis: price response under different fuel/temperature paths.
- Anomaly detection on price and volume using VAE.

8.2 Indian onion price + weather (Shanthini et al.)

8.2.1 Data fields

- Onion market price (daily; Kurnool).
- Maximum and minimum temperature.
- Humidity.
- Precipitation and probability of precipitation.
- Wind speed.
- Sea level pressure.
- Solar radiation.
- UV index.

8.2.2 Primary tasks

- One-step-ahead onion price prediction.

8.2.3 Secondary tasks

- Model comparison: tree-based models vs k-NN vs DL.
- Feature importance: quantify the role of each weather variable.
- Benchmark for Vietnam coffee dataset design (similar structure, richer geography).

8.3 Indian vegetable price panel (Prashantha et al.)

8.3.1 Data fields

- Date / timestamp.
- State, district, market identifiers.
- Commodity and variety (multiple vegetables).
- Minimum, maximum and modal prices.
- Arrival date / arrivals quantity

8.3.2 Primary tasks

- Cross-market price prediction for key vegetables.

8.3.3 Secondary tasks

- Market–crop allocation: identify which markets demand which crops more.
- Cross-sectional generalization: train on some markets, test on others.
- Baseline for comparing econometric, tree-based and DL models.

8.4 Template feature-rich cereal dataset

(inspired by the 3 review papers)

8.4.1 Target crops

- Wheat.
- Corn.
- Rice.

8.4.2 Data fields (typical in recent SLR studies)

- Historical spot prices and/or futures prices.
- Production, yield and stock levels.
- Weather: temperature, rainfall, drought indices.
- Cost and macro variables: fuel prices, exchange rates, trade volumes, policy dummies.

8.4.3 Primary tasks

- Multi-crop price prediction (joint or per-crop models).
- Model family comparison: ARIMA / SARIMAX vs tree-based vs DL vs hybrid.

8.4.4 Secondary tasks

- Decomposition + model pipelines (e.g., CEEMDAN–TDNN, LSTM–GARCH).
- Volatility / risk modelling alongside point forecasts.

9 Feature Engineering

9.1 Feature families

9.1.1 Temporal features

- Date or timestamp.
- Arrival date.
- Year.
- Lagged prices

9.1.2 Geographical features

- State.
- District.
- Market.

9.1.3 Commodity & market features

- Commodity and variety.
- Minimum price, maximum price, modal price, target price, general price levels.
- Cross-commodity prices (e.g., soybean oil/meal → soybean; wheat → corn).
- Yield, supply, demand, production.
- Beginning stocks, ending stocks, total use, consumption-to-use ratio.
- Production value at producer price.

9.1.4 Weather & environmental features

- Maximum and minimum temperature, humidity.
- Precipitation, probability of precipitation, rainfall.
- Wind speed, sea level pressure.
- Solar radiation, ultraviolet index.
- Sun elevation, sun azimuth.
- Cloud cover, including satellite-derived cloud cover from Google Earth Engine.

9.1.5 Energy, cost & resource features

- Fuel price, energy cost.
- Land use.
- Availability of human resources.
- Subsidies, taxes.

9.2 Handling missing and noisy data

9.2.1 Handling missing data

- Backward fill.
- Forward fill
- Linear regression imputation.
- RandomForest imputation
- KNN-based imputer.
- Dropping missing observations (when safe).
- Average of previous and next price.

9.2.2 Handling noisy / inconsistent data

- Interquartile distance (IQR) method for outlier detection.
- Discard duplicate values.
- Unify prices (consistent units, currencies, and scaling).

9.3 Time-series preprocessing and statistical transforms

9.3.1 Stationarity and differencing

- Assess stationarity using ADF and KPSS tests.
- Number of differences determined by ADF test results.
- First difference of the series when needed.

9.3.2 Lag and seasonality selection

- Pick lags using information criteria (AIC, BIC).
- Use PACF and ACF plots for lag selection.
- Dynamic harmonic regression using Fourier terms as regressors for seasonality.

9.3.3 Scaling and normalization

- Min–max scaling.
- robust scaler
- FA pipeline: log $\rightarrow$ deseason $\rightarrow$ detrend $\rightarrow$ standardize.
- FD pipeline: first difference $\rightarrow$ standardize.
- TA/SA/UN variants as alternative preprocessing pipelines.
- Rolling windows for deep learning models.
- Ensure stationarity for VAR / ARIMA-type models.

9.4 Model-specific pipelines and practical implementation steps

9.4.1 Evolutionary + neural approaches

- DE: differential evolution for lag selection and weight initialization.
- Backpropagation fine-tunes neural network weights after DE initialization.

9.4.2 Tabular market data workflows

- Load the market table and use / derive modal price from provided price fields.
- Encode categorical columns (state, market, commodity, variety) before feeding into ML models.

9.4.3 Price–weather fusion workflows

- Clean and prepare the merged price–weather table.
- Handle missing weather entries explicitly.
- Use standard train/test splits for model comparison.

9.4.4 Decomposition and temporal splits

- Decompose the time series before modeling, especially for volatile staple crops.
- Use time-ordered splits: train on the past, test on the future as the standard evaluation protocol.

10 Experimental Results

This section presents the experimental results from each team member's model implementations on wheat futures price forecasting using FRED-MD macroeconomic features.

10.1 Nishant: Attention-Based CNN/RCNN Models

Table 7: CNN/RCNN Model Performance Comparison on Test Set (Nishant)

Model	RMSE	MAE	R^2	Rank
green!15 CNN with Self-Attention	**78.67**	**59.09**	**0.765**	**1**
RCNN with Attention	88.02	68.06	0.705	2
CNN with Attention	148.49	116.76	0.162	3
RCNN with Self-Attention	155.01	115.93	0.087	4

10.2 Nived: LSTM and Econometric Models

Table 8: LSTM and Econometric Model Performance Comparison on Test Set (Nived)

Model	MSE	RMSE	MAE	R^2
[Add results here]	–	–	–	–

10.3 Richard: GRU Models and Naive Forecaster

Table 9: GRU Model Performance Comparison on Test Set (Richard)

Model	MSE	RMSE	MAE	R^2
Baseline GRU	1083.87	32.92	20.08	0.959
GRU with Attention	5599.28	74.83	56.62	0.787
Bi-GRU	2345.12	48.43	42.75	0.911
Bi-GRU with Attention	1498.24	38.71	25.41	0.943
Naive	414.96	20.37	12.85	0.984

10.4 Peilin: ESN/ELM Models

Table 10: ESN/ELM Model Performance Comparison on Test Set (Peilin)

Model	MSE	RMSE	MAE	R^2
[Add results here]	–	–	–	–

10.5 Peter: Reinforcement Learning Models

Table 11: RL Model Performance Comparison on Test Set (Peter)

Model	CAGR	Sharpe Ratio	Max Drawdown	Win Rate
[Add results here]	–	–	–	–

11 Author Contributions

11.1 Peilin Yao

- 1 Organized and rewrite the introduction, purpose and scope
- 2 add descriptions to dataset categories
- 2.3 Data platforms
- 5.1.2 ANN, ELM, RNN, ESN
- 5.1.2 GRU vs LSTM table
- Implemented the following models in codes:
  - ESN
  - ELM
  - GA-ELM

11.2 Richard

- 1.2 Scope of the report (commodity markets, importance of agricultural commodities)
- 2.1.1.c Futures vs. spot market differences explanation
- 2.1.4 Perishable vs. Storable commodities
- 2.2.2 Remote sensing
- 2.3.2.d Other data sources
- 3.2.2 Remote-sensing and satellite-derived features

- 4.3.3.a VAE-based anomaly detection
- 5.1.3 VAE model mechanism, Transformers mechanism
- 5.2.1 ARIMA and Seasonal ARIMA
- 5.2.3 GARCH, GJR-GARCH and MEM
- 5.3.2 Decision Trees, XGBoost, LightGBM, CatBoost, Strengths and Weaknesses
- 5.5.2 Hybrid Models: (Bi)LSTM-VAE, LSTM-SARIMA, LSTM-GARCH
- 5.5.4 Competitive Ensemble Model
- Implemented the following models in codes:
  - GRU
  - GRU with attention
  - Bi-GRU
  - Bi-GRU with attention
  - GRU with attention and skip connection
  - Naive Forecaster

11.3 Peter

- 2.1.2.a Indian vegetable prices
- 2.2.1.a Local weather for Indian markets
- 2.3.1.a Systematic review databases
- 2.3.2.c Public datasets
- 2.3.3.b Environmental data
- 2.3.4 Typical ranges
- 3.1.1.a Spot and wholesale prices
- 3.2.1.a Daily weather for Indian markets
- 4.1.2.a Spot and wholesale market prices
- 4.3.1.a Market-crop allocation support
- 5.4 Other machine-learning models
  - 5.4.1 k-Nearest Neighbors (k-NN)
  - 5.4.2 Linear Regression
  - 5.4.3 Bayesian Regression
  - 5.4.4 Support Vector Machines (SVM) and Support Vector Regression (SVR)
- Implemented the following models in codes:
  - PPO
  - SAC

11.4 Nishant

- **Survey Sections:**
  - 2.1.1.a,b Daily agricultural futures price series and multi-asset panel data
  - 2.1.3 USDA production and balance-sheet data
  - 2.2.3 Supply–demand fundamentals (production and stock metrics)
  - 2.2.4 Cross-commodity and price spillovers
  - 2.2.5 Macroeconomic and financial indicators
  - 2.2.6 Time-series transformations
  - 2.3.2.a Futures and derivatives data sources
  - 2.3.3.a USDA portals
  - 3.1.1.b,c,d,e Futures price series, multi-asset panel, annual fundamental-price series
  - 3.1.2 Futures contract metadata
  - 3.1.3.b,c Continuous futures construction, stationarity transformations
  - 3.4 Feature selection and lag determination methodologies
  - 4.1.1.b Multi-horizon forecasting
  - 4.1.2.b,c Futures contract prices, ensemble and composite forecasts
  - 4.2.a,d Yield forecasting pipelines, structural supply-demand-price pipelines
  - 4.3.1.b,c,d,e Market selection, agricultural timing, trading/risk management, farmer profit/loss
  - 4.3.2.a Policy intervention and planning
  - 5.1.2 CNNs for time series, Attention-based RNN/CNN variants (full mathematical formulations)
  - 5.1.3 Extended VAE mathematical formulation, Transformer mathematical formulation
  - 5.2.2 SARIMAX formulation and pros/cons, VAR general formulation and pros/cons
  - 5.2.3 GARCH stationarity conditions and pros/cons, GJR-GARCH leverage effect, MEM error distribution
  - 5.2.4 EMD/EEMD/CEEMD/CEEMDAN mathematical formulations and pros/cons
  - 5.2.5 Econometric model comparison table
  - 5.3.2 Decision tree mathematical formulation and pros/cons, Random Forest formulation, XGBoost/LightGBM/CatBoost pros/cons, computational complexity table
  - 5.5.3 STL-LOESS decomposition formulation, multi-stage hybrid pipelines
  - 5.5.4 Ensemble combination strategies (simple average, weighted average, stacking)
  - 5.5.5 Hybrid approaches summary table

  - 6 Underlying Model Assumptions (entire section)
  - 7.1.1 Evaluation metrics mathematical formulations (RMSE, MAE, MAPE, MASE, RRMSE) and interpretations
  - 7.1.2 Metric selection guide table and practical recommendations
- **Implemented and benchmarked the following models in code:**
  - CNN with Self-Attention
  - CNN with Temporal Attention Pooling
  - RCNN with Temporal Attention Pooling
  - RCNN with Self-Attention
- **Other contributions:**
  - Image selection and captioning for model architecture diagrams
  - LaTeX formatting and document compilation
  - Mathematical formulation standardization across sections

11.5 Nived

- Contributed to several sections of the Survey
- Contributed to datasets, scope, purpose, hybrid models, deep learning models, feature engineerin
- Contributed to econometric models, EMD based approaches, hybrid models, deep learning mode
- Implemented and benchmarked the following models with the code:-
  - LSTM without Attention
  - LSTM with Attention
  - LSTM with Skip Connections
  - Bi-LSTM without Attention
  - Bi-LSTM with Attention
  - ARIMA
  - SARIMAX